İşletim Sistemleri
(Operating Systems)

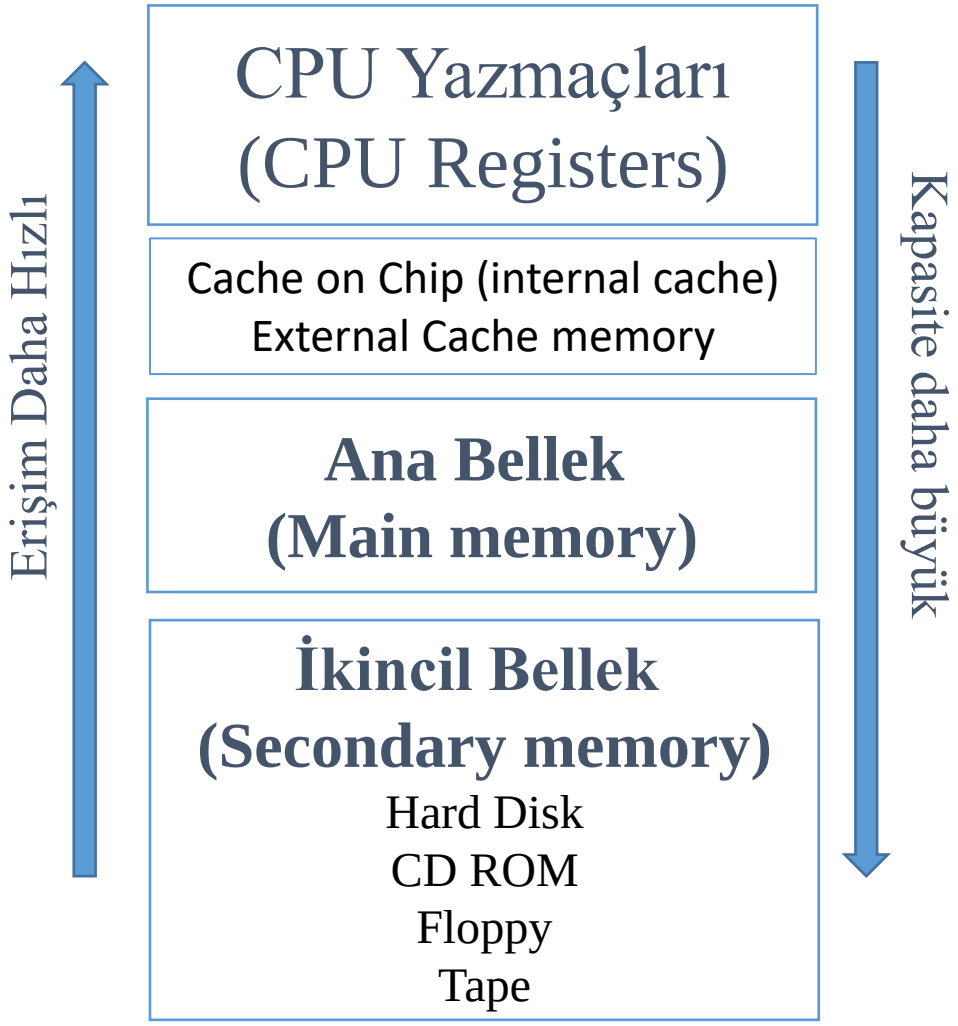
Bellek Yönetimi
(Memory Management)

Bellek Yönetimi (Memory Management)

- Bilgisayar sistemlerinde sınırlı miktarda bellek bulunmakta ve çalışan her program bir miktar belleğe ihtiyaç duymaktadır.
- İşletim sistemi bu belleği ne kadar verimli kullanırsa ve process'leri ne kadar düzgün yerleştirirse belleğin içerisine sığabilen process miktarı o kadar fazla olur.
- Bellek verimli ve dikkatli bir şekilde yönetilmesi gereken önemli bir kaynaktır.
- Yazılım teknolojileri, bellek teknolojilerinden daha hızlı büyüyor, bu nedenle verimli bellek yönetimi her zaman gerekli olacak.

Bellek Organizasyonu

- Von Neumann mimarisine sahip bir bilgisayar sisteminde bellek sistemi en az üç seviyeden oluşur; en üst seviyede CPU yazmaçları (registers), orta seviyede ana bellek ve en alt seviyede de ikincil bellektir.
- Yazmaçlara CPU'nun bir saat çevriminde ulaşması mümkündür. Ana belleğe erişim ise birçok CPU saat çevrimi gerektirir.
- Ana bellek ile CPU arasına çok daha hızlı ve CPU'ya yakın bir saklama alanı oluşturulur (cache memory).
- Bir programı çalıştırmak için ikincil bellekten ana belleğe getirilmesi gerekir.

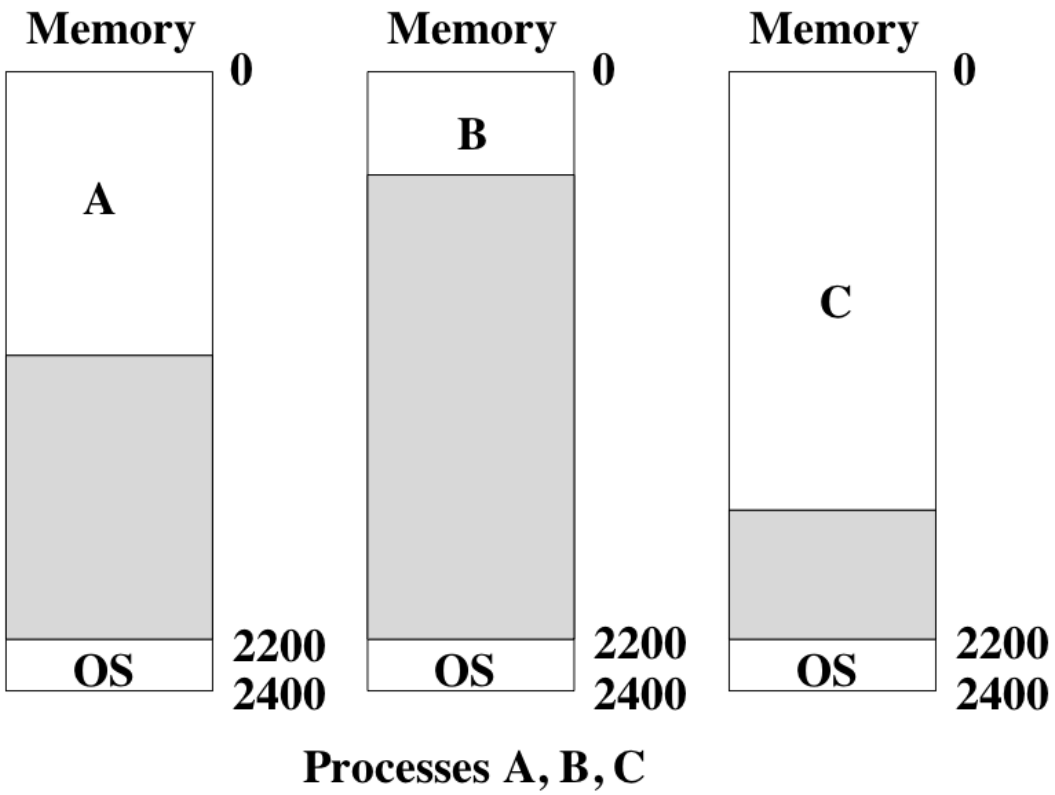


Mono programming ve Bellek Yönetimi (Swapping veya Paging olmadan)

- En basit bellek yönetimi şeması, bir program ve işletim sistemi arasında belleği paylaştırarak bir kerede yalnızca bir programı çalıştırmaktır. Sistem bu şekilde düzenlendiğinde, aynı anda yalnızca bir process çalışıyor olabilir.
- Mono programming özelliğindeki bu en basit bellek yönetim planları günümüz masaüstü ve dizüstü bilgisayarlardaki işletim sistemlerinde artık kullanılmamasına karşın,
- bu yöntemler bazı avuçiçi sistemler (palmtop), gömülü (embedded) sistemler ve akıllı kart (smart card) sistemlerinde hala kullanılmaktadır.

Mono programming ve Bellek Yönetimi

- Örn. DOS (Disk Operating Sytem)



Multi programming ve Bellek Yönetimi

- Basit gömülü sistemler dışında, mono programlama artık pek kullanılmıyor.
- Modern işletim sistemleri, eşzamanlı olarak birden fazla process'i çalıştırmaya izin verir.
- İki veya daha fazla process aynı anda ana bellekte bulunduğunda işlemcinin process'ler arasında paylaşılması, multi programming olarak adlandırılır. Multi programming CPU kullanımını arttırır.
- **Fakat, multi programming için bellekte birçok process'i barındırmak için bellek yönetimi gereklidir.**

Bellek Yönetimi (Memory management)

- İşletim sisteminin birincil görevi temel sistem kaynakları olan işlemci (CPU) ve ana belleği (main memory), process'ler arasında paylaştırmaktır.

- Process'lerin yürütülebilmesi için bellekte bulunması gerekir.

- CPU'nun process'ler arasında paylaşımı, ana bellek paylaşılmaksızın gerçekleşemez.

- Belleğin process'ler arasında paylaştırılmasına bellek yönetimi **(memory management)** adı verilir.

Bellek Yönetimi (Memory management)

- İşletim sisteminde fiziksel bellek hiyerarşisinin yönetiminden sorumlu olan birime de **bellek yöneticisi (memory manager)** adı verilir.

- Bellek yöneticisi, process'lerin hangi bellek bölümüne yerleşeceğine karar verir, bellek erişimini ve yerleşimini denetler, process'ler arasında bellek paylaşımını organize eder.

Bellek Yönetimi (Memory management)

- Bellek yöneticisinin başlıca görevleri:
- Process'lere bellek tahsis etmek (allocate), tahsis edilen belleği geri almak (deallocate),
- Belleğin hangi bölümlerinin kullanımda olduğunu, hangi bölümlerinin (bitişken alanların) kullanılmadığını izlemek,
- Bellek ile disk arasındaki yer değiştirme (swap) işlemlerini gerçekleştirmek.
- Bellek Yönetim Birimi (Memory Management Unit-MMU) donanımı yardımıyla mantıksal adresleri fiziksel adreslere dönüştürmek.

Bellek Bölümleme (Memory Partitioning)

- Modern işletim sistemleri eşzamanlı olarak birden fazla process'in yürütülmesine olanak verir. Ana bellek, birden fazla process arasında paylaştırılmalı ve bölünmelidir.
- Bunu yapabilmek için kullanılan yöntemler:

Bitişik bellek ayırma (Contiguous memory allocation)

- Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)
- Dinamik Bölümlü Bellek Yönetimi (Dynamic partitioning)

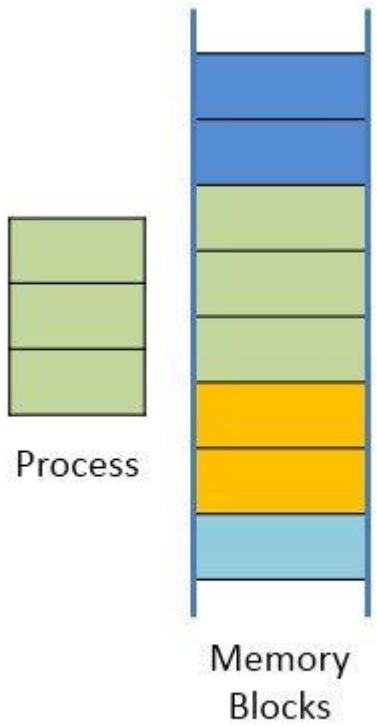
Bitişik olmayan bellek ayırma (Non-Contiguous memory allocation)

- Sayfalı Bellek Yönetimi (Paging)
- Kesimli Bellek Yönetimi (Segmentation)

- *Fixed partitioning ve Dynamic partitioning gibi basit bellek yönetim planları günümüz masaüstü ve dizüstü bilgisayarlardaki işletim sistemlerinde artık kullanılmamaktadır.*

Bitişik bellek ayırma (Contiguous memory allocation)

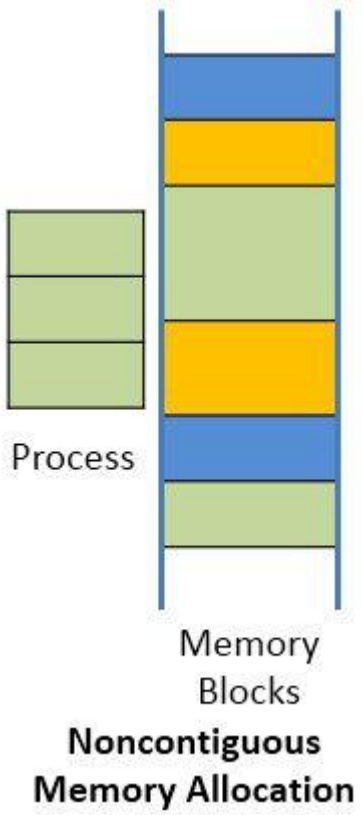
Değişmez (Fixed) ve Dinamik (Dynamic) Bölümlü Bellek yönetimleri, bir process'in fiziksel adres alanının bitişik olmasına izin veren iki yöntemdir.



Contiguous Memory Allocation

Bitişik olmayan bellek ayırma (Non-Contiguous memory allocation)

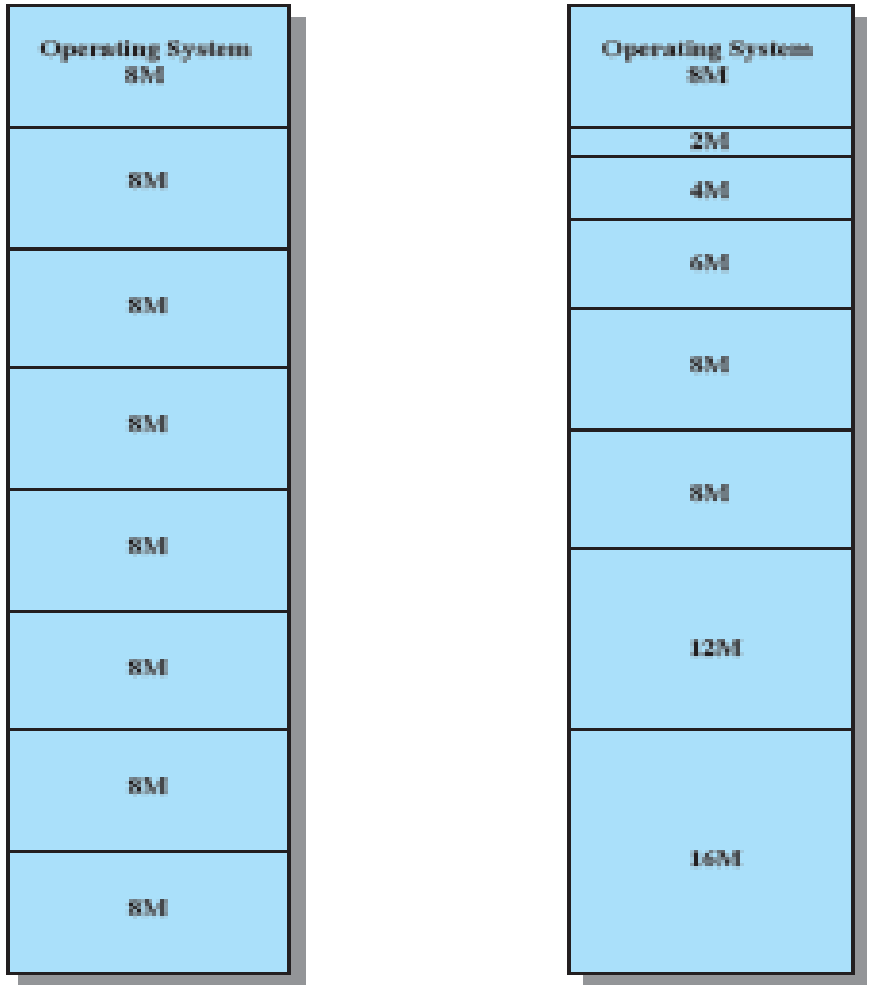
Sayfalama (Paging) ve Segmentasyon (Segmentation), bir process'in fiziksel adres alanının bitişik **olmamasına** izin veren iki yöntemdir.



Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)

- İşletim sisteminde ana belleğin bir bölümü işletim sisteminin kendisine ayrılmıştır.
- Geri kalan bellek alanı ise bellek yöneticisi tarafından process'ler arasında paylaştırılır.
- Bu, bellek yönetiminin en basit şeklidir.

- Bellek alanı
- **eşit (equal-size partitioning)**
- veya
- **eşit olmayan (unequal-size partitioning)**
- uzunlukta bölümlere ayrılabilir.



(a) Equal-size partitions (b) Unequal-size partitions

Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)

- Bellek **çok sayıda eşit veya eşit olmayan boyutta** bölüme ayrılabilir ve her bölüm bir process'i içerebilir.
- Process'in hangi bölüme yerleştirileceği fark etmez.
- Process'lerin adres alanlarının farklı olması sağlanır, bir process başka bir process ile veya sistemle karıştırılamaz.
- Multi programming sistemlerde eşzamanlı çalışan process sayısı ayrılan bölüm sayısına bağlıdır.
- Bir bölüm boşaldığında, başka bir process seçilerek bölüme atanır.

Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)

- **Eşit uzunlukta bölümleme (Equal-size partitioning) :**
- Değişmez bölümlü bellek yönetiminde bölümler **eşit uzunlukta** (equal-size) olabilir.
- Bellek sistem başlangıcında sabit sayıda bölüme ayrılır. Bölümlerin boyutu ve sayısı önceden tanımlanmıştır ve sonradan değiştirilemez.
- Her bölüm bir process'i içerebilir ve process'in hangi bölüme yerleştirileceği fark etmez.
- Maksimum aktif process sayısı sınırlı.
- İşletim sistemi tarafından gerçeklenmesi kolay.
- Getirdiği ek yük az.

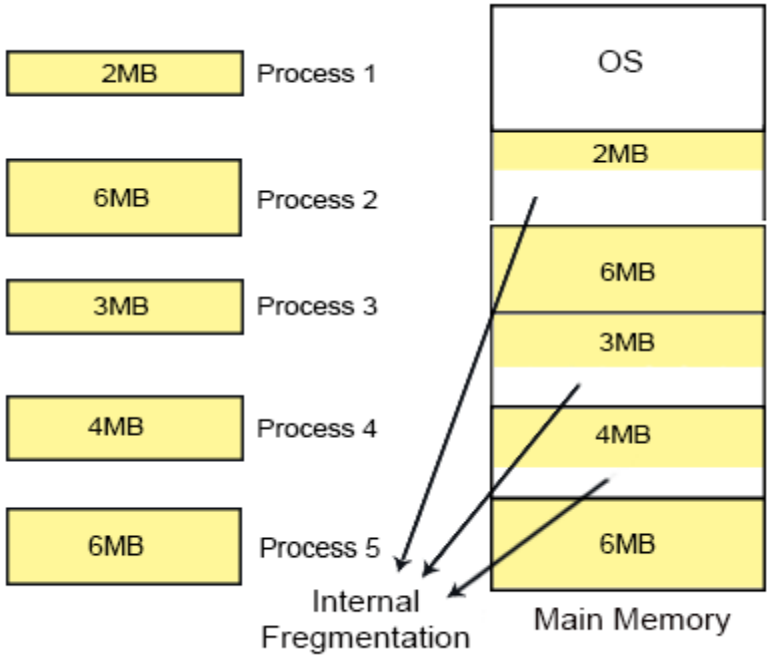
Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)

- **Eşit uzunlukta bölümleme (Equal-size partitioning) :**

- 1. Bir process birden fazla bölümde bulunamaz.
- Eğer process boyutu bölüm boyutundan büyükse, o process ana belleğe yüklenemez.
- Eğer RAM'de 10 KB ve 10 KB'lık iki boş bölüm varsa ve gelen process boyutu 20 KB ise, ana bellek bunu barındıramaz.
- 2. Bir bölümde yalnızca bir process olmalıdır çünkü bir bölüme birden fazla process yüklenemez.

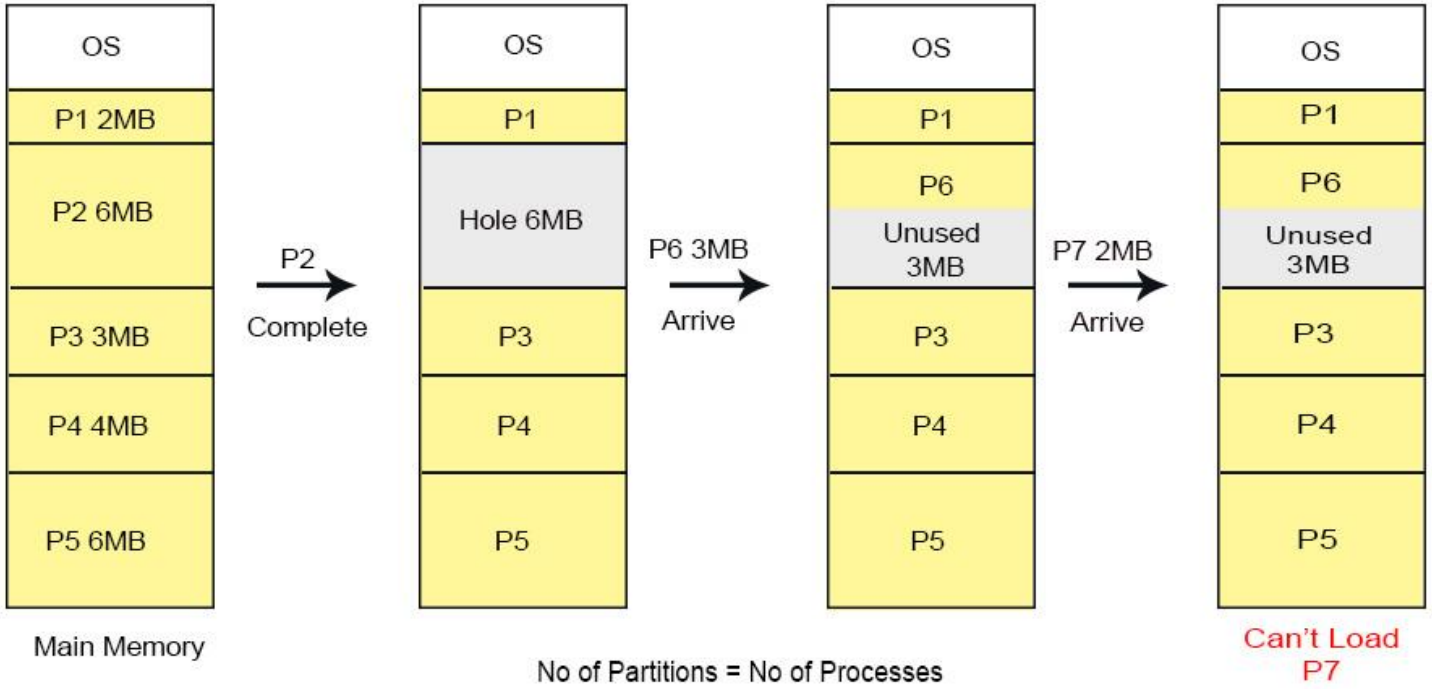
Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)

- **Eşit uzunlukta bölümleme (Equal-size partitioning) :**
- 2MB, 6MB, 3MB, 4MB ve 6MB uzunlıgunda beş adet process olduğunu varsayalım.

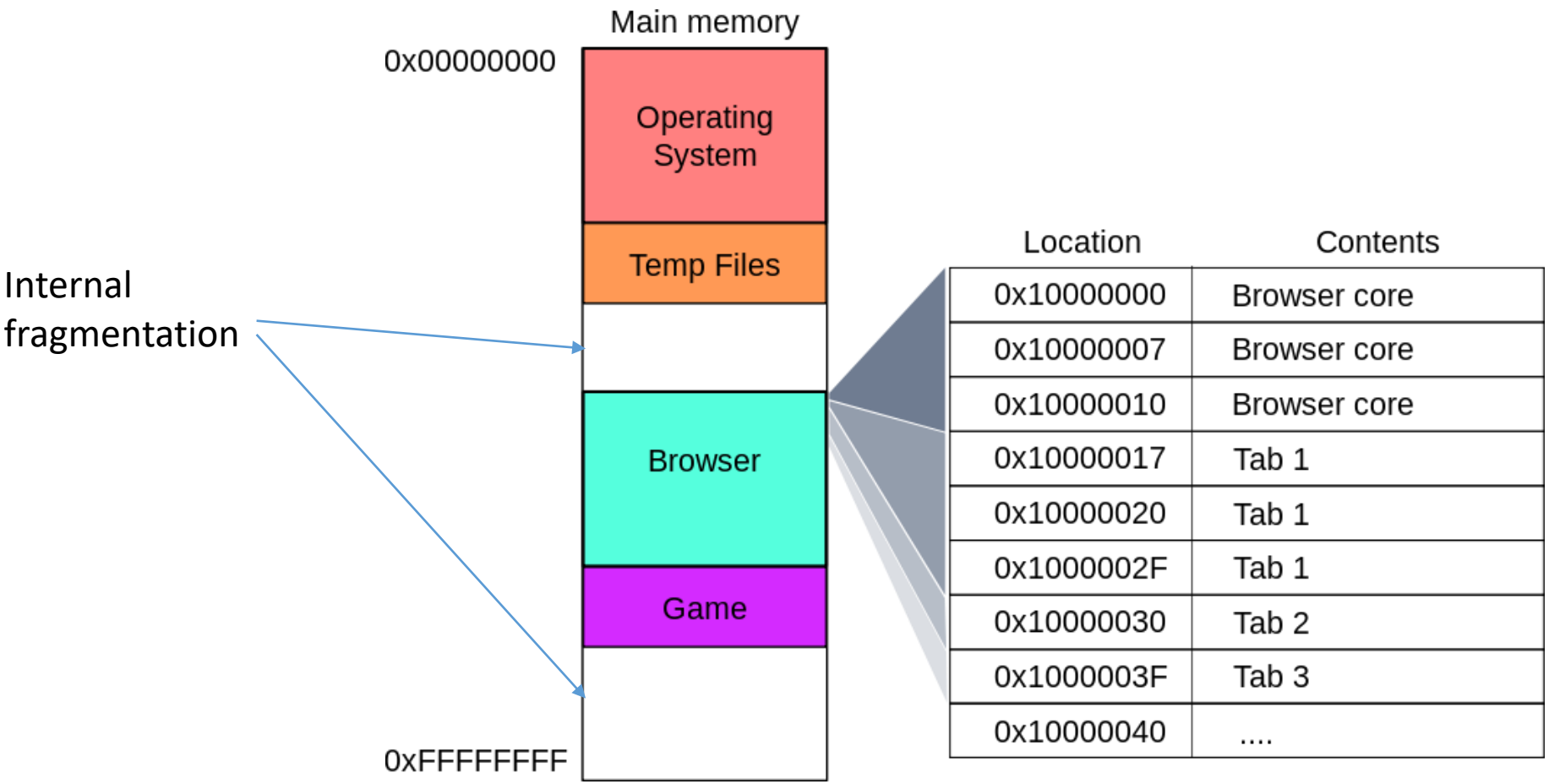


Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)

- **Eşit uzunlukta bölümleme (Equal-size partitioning) :**
- P7 geldiğinde, her bölüm bir process içerir. Bu nedenle, P7 iç parçalanma biciminde 3MB alan mevcut olsa bile belleğe yüklenemez.



Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)
Equal-size partitioning



Değişmez bölümlü bellek yönetiminde bölümler sabit bir büyüklükte (equal-size) olduğundan Bellek kullanımı etkin değildir;

Her process ne kadar boyu küçük de olsa tam bir bölmeyi elinde tutar.

⇒ iç parçalanma (internal fragmentation)

Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)

Eşit uzunlukta bölümleme (Equal-size partitioning) :

Değişmez bölümlü bellek yönetiminde bölümlerin eşit bir büyüklükte olması durumunda her process ne kadar boyu küçük de olsa tam bir bölmeyi elinde tuttuğu için iç parçalanma (internal fragmentation) olayına sebebiyet vermektedir.

Dolaysıyla bellek kullanımı verimli değildir.

Ayrıca, boyutu fiziksel bellek bölümünün gerçek boyutundan daha büyük olan bir process belleğe yüklenemez.

Bu durumda;

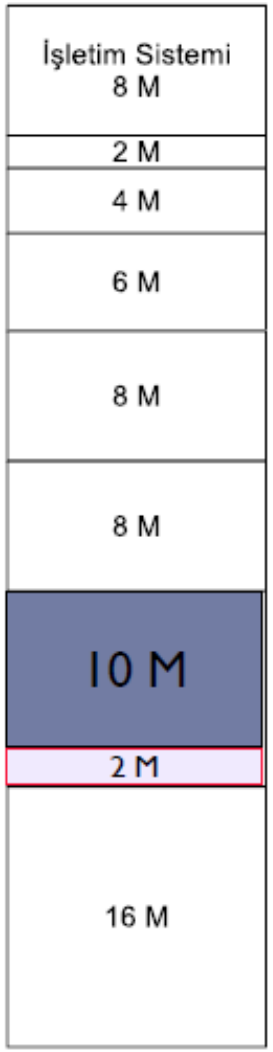
eşit boyda olmayan bölümler kullanılması sorunu bir derece çözer.

Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)

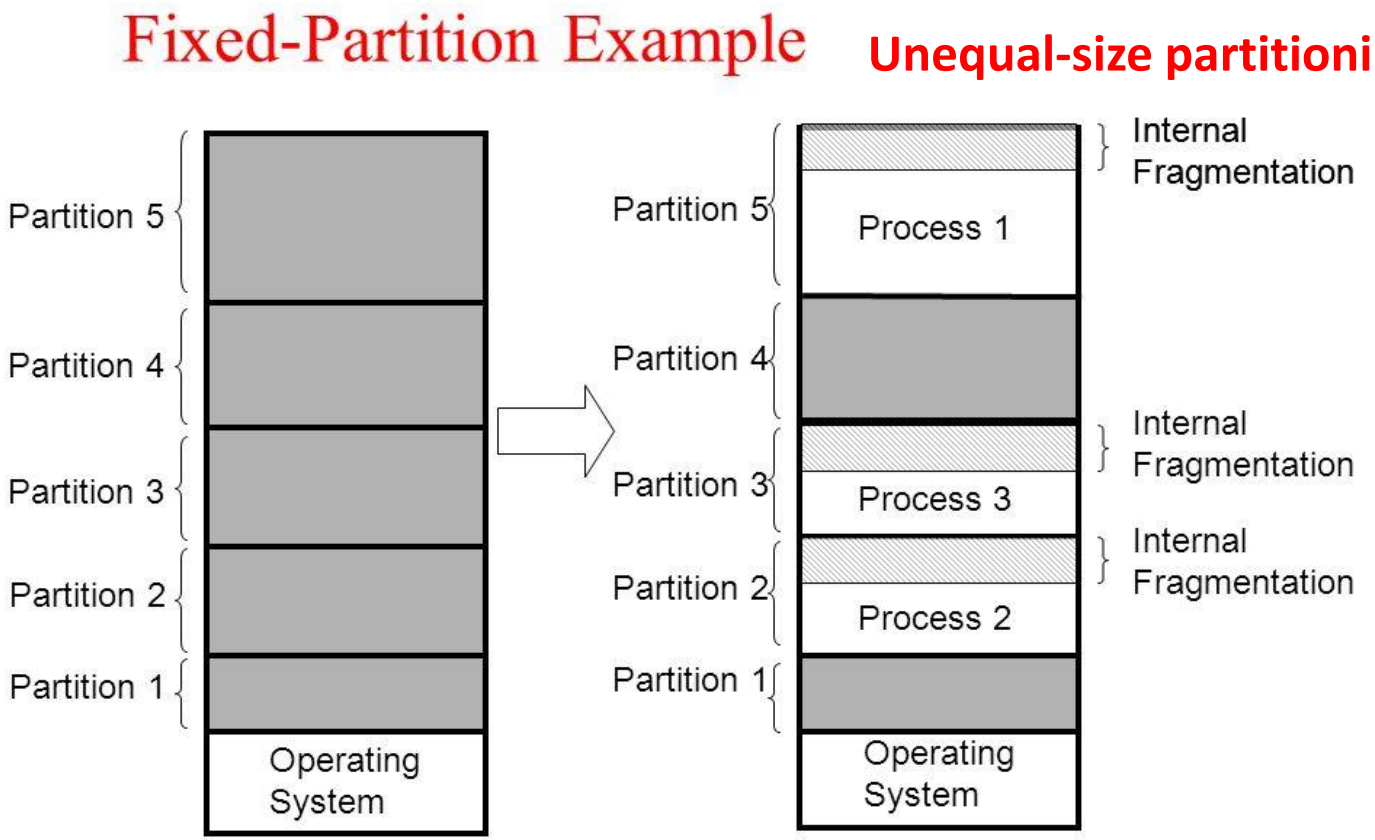
- **Eşit olmayan uzunlukta bölümleme (Unequal-size partitioning) :**

- Değişmez bölümlü bellek yönetiminde bölümler **farklı büyüklükte (Unequal-size)** de olabilir.

- Bölüm içi boş kalan yer miktarını en aza indirmek üzere her process sığacağı en küçük bölmeye yerleştirilir.
- Toplam büyüklüğü 64 MB, bölüm boyutları da farklı olan bir ana bellek yan tarafta görünmektedir.
- 10 MB bir process istek yolladığında en az iç parçalanmanın oluşacağı bölüm bulunacak process bu bölüme konulacaktır.



Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)



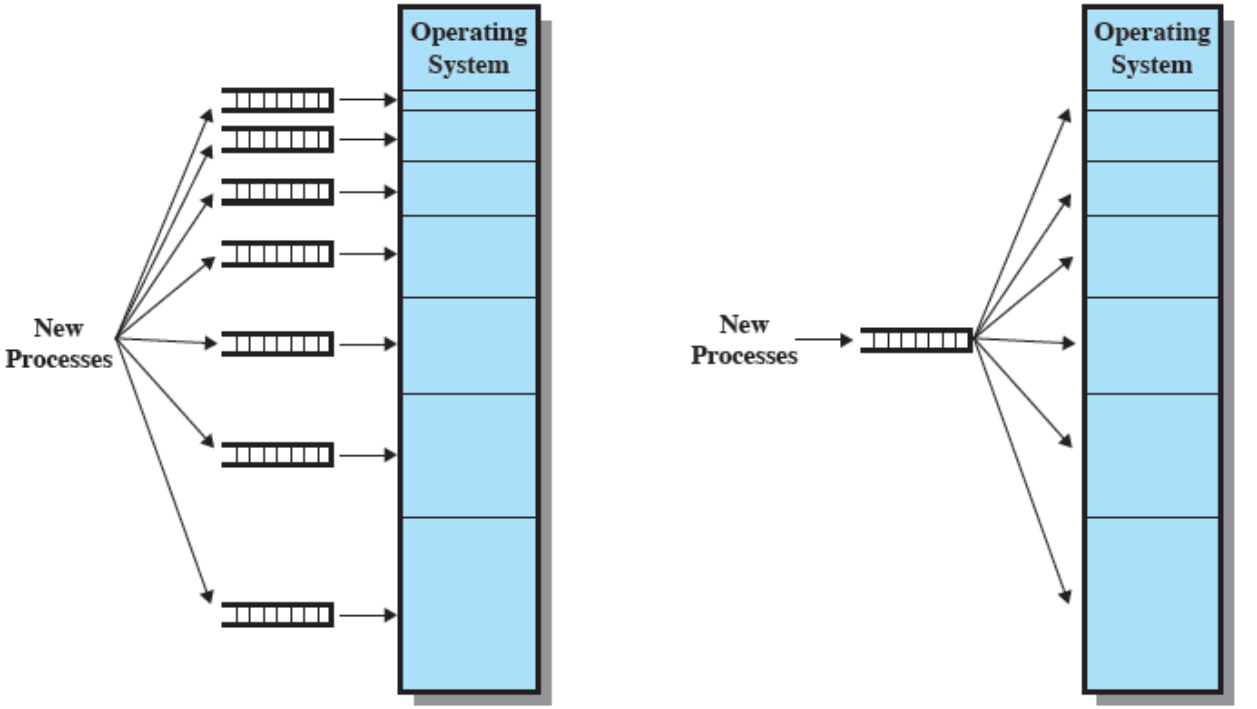
Unequal-size partitioning :

Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)

- Farklı büyüklükteki bölüm boyları olduğu durumda;
- Her process sığacağı en küçük bölüme yerleştirilir.
- Ayrılan her bölüm için bir process kuyruğu veya tüm bölümler için tekbir process kuyruğu bulunur.
- Ayrılan her bölüm için bir process kuyruğu bulunduğu durumda, bir process geldiği zaman kendisini tutabilecek olan en küçük boyutlu bölümün giriş kuyruğuna eklenir.
- Bölüm içi boş kalan yer miktarını en aza indirmek amaçlanır.
- Bir process eklendiği bölümün hepsini kullanmaz ise bu kullanılmayan bellek boşuna kullanılmış ve israf edilmiş olur.

Değişmez Bölümlü Bellek Yönetimi (Fixed partitioning)

- Gelen processler boyutlarına göre kuyruklara eklendiğinde, büyük boyutlu bellek bölgelerinin kuyrukları boş kalıp kullanılmaz iken, küçük boyutlu bellek bölgelerin kuyrukları dolu olabilir.
- Kuyruk dolu olduğunda gelen process boş yer olmasına rağmen bekler.
- Bunun yerine tüm processler bir kuyruğa konulur ve sıraları geldiğinde uygun bellek gözüne konulurlar.



(a) One process queue per partition

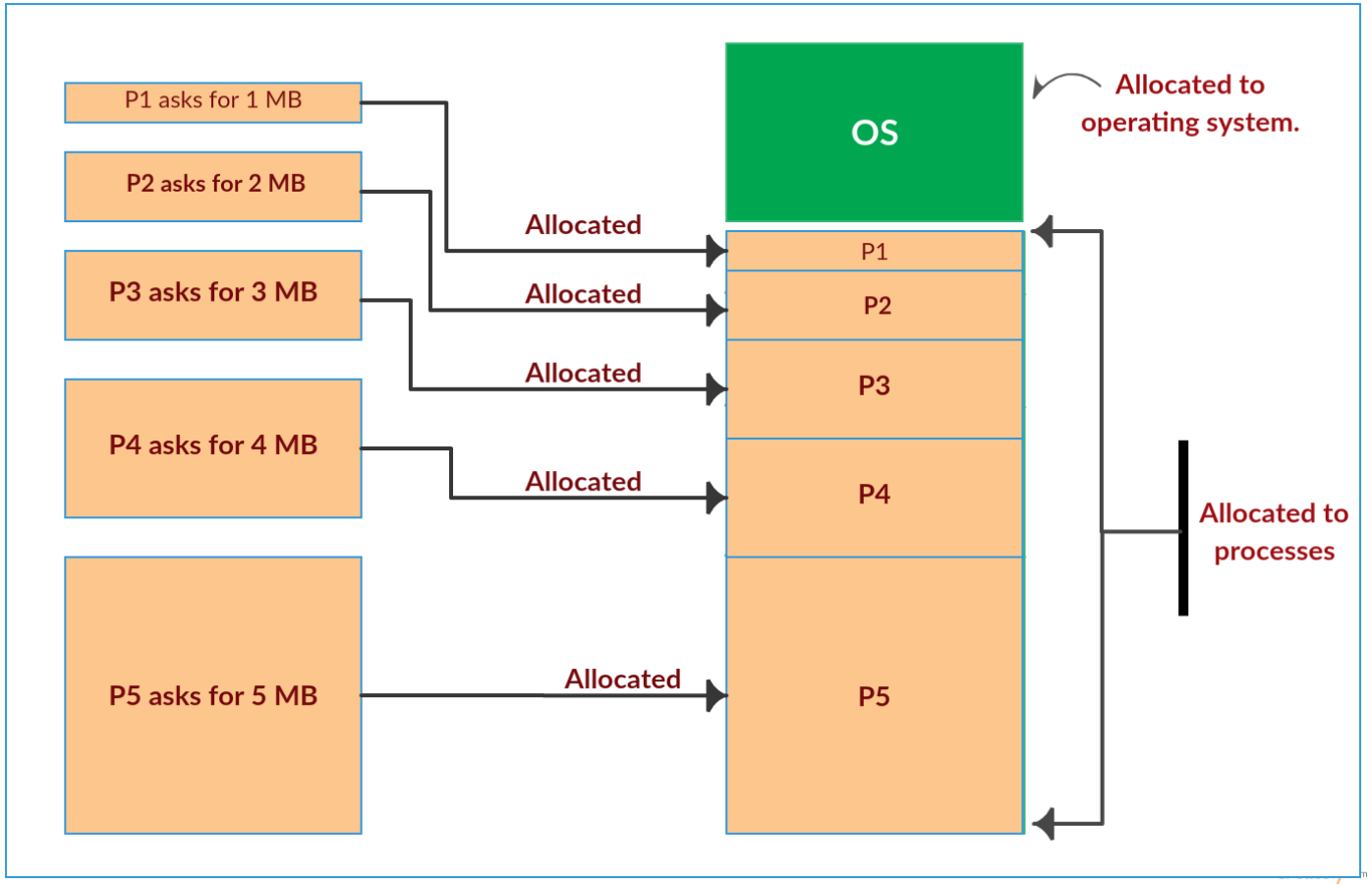
(b) Single queue

Dinamik Bölümlü Bellek Yönetimi (Dynamic Partitioning)

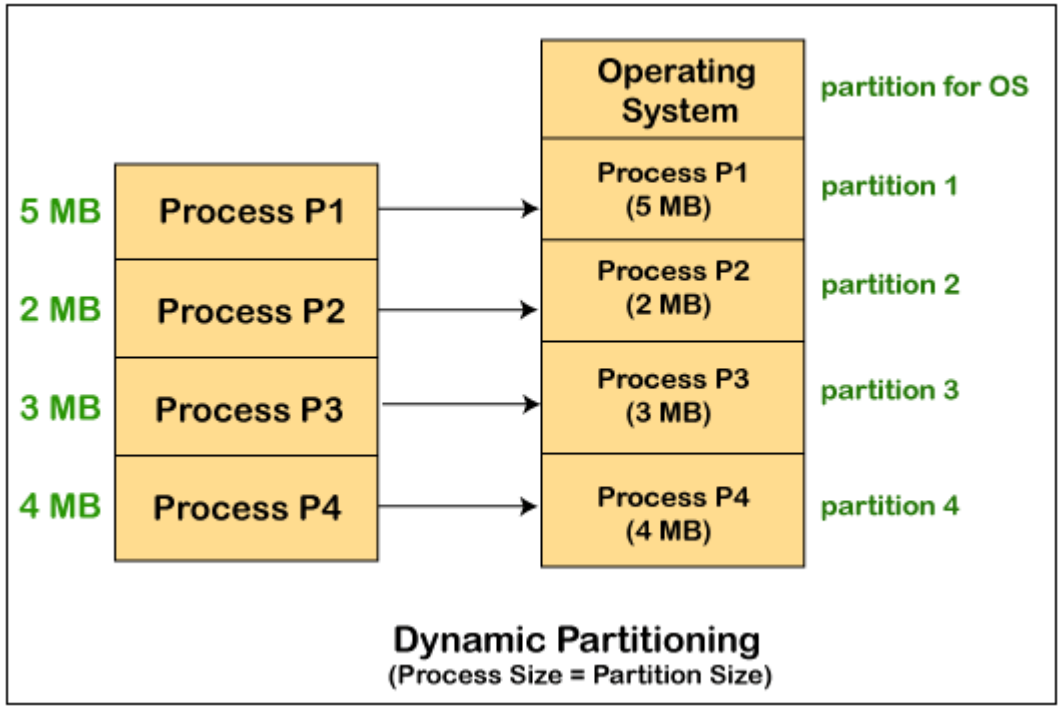
- Bu yöntemde bölme sayısı ve bölme boyları sabit değildir.
- Process'lere sadece gerektiği kadar bellek alanı atanır.
- İşletim sistemi belleğin boş ve dolu olan bölümlerini bir tabloda tutar.
- Bölümlerle ilgili, bölüm başlangıç adresi, bölüm boyu gibi bilgiler tutulmaktadır.
- Bu yöntemde process'lere atanan bölümlerin yanı sıra, bu bölümler arasında kalan boş alanların da izlenmesi gereklidir.

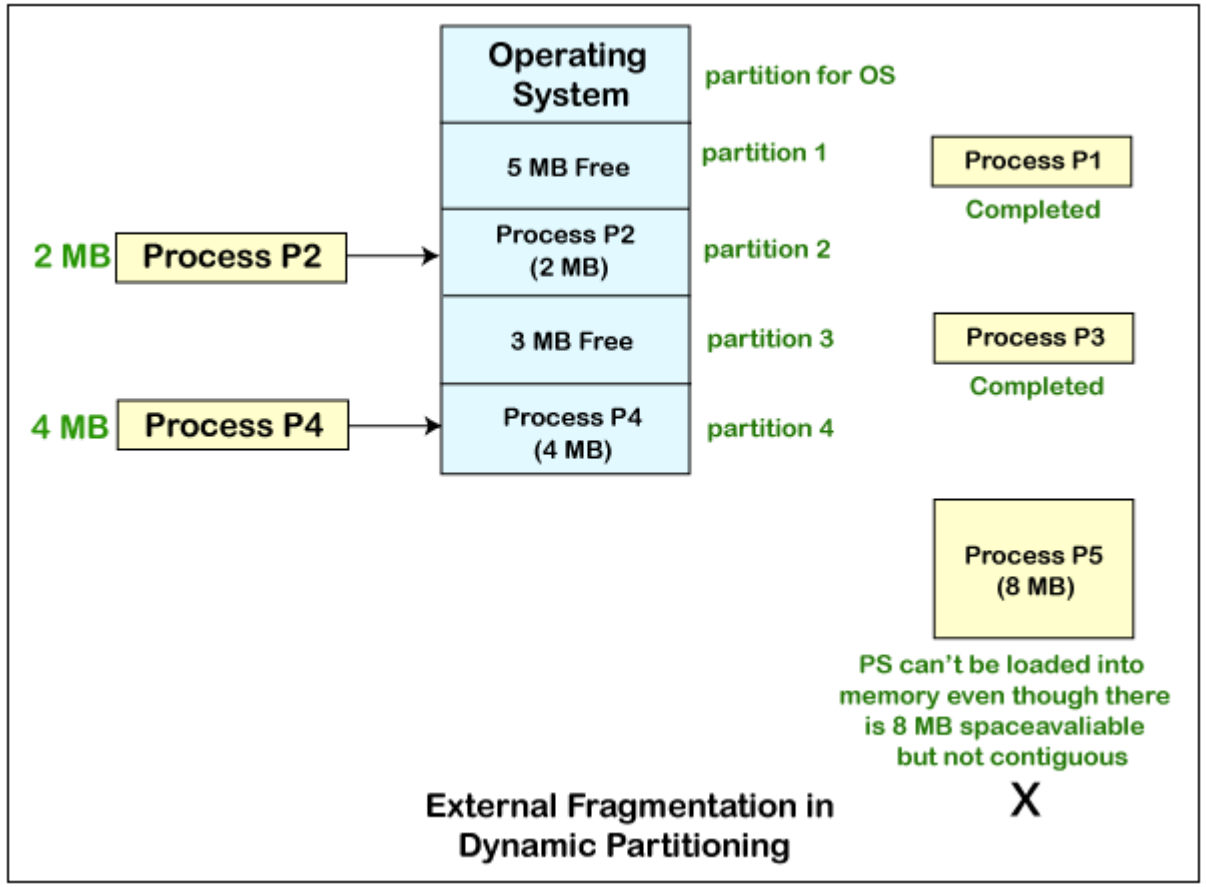
- Kullanılmayan boş yerler yine de oluşur. Dış parçalanma (external fragmentation).

Dinamik Bölümlü Bellek Yönetimi (Dynamic Partitioning)



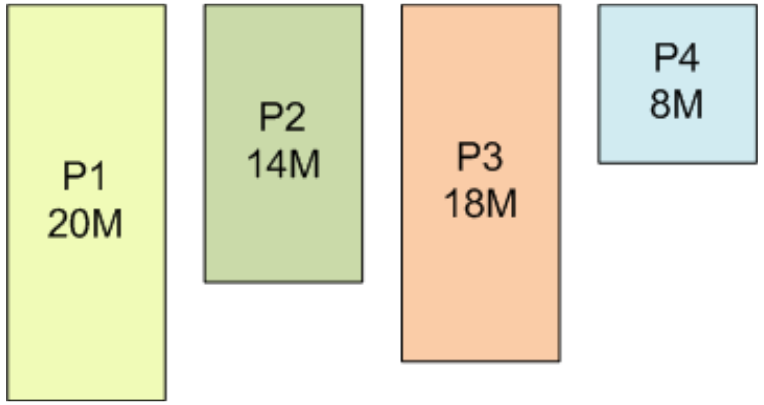
Dinamik Bölümlü Bellek Yönetimi (Dynamic Partitioning)





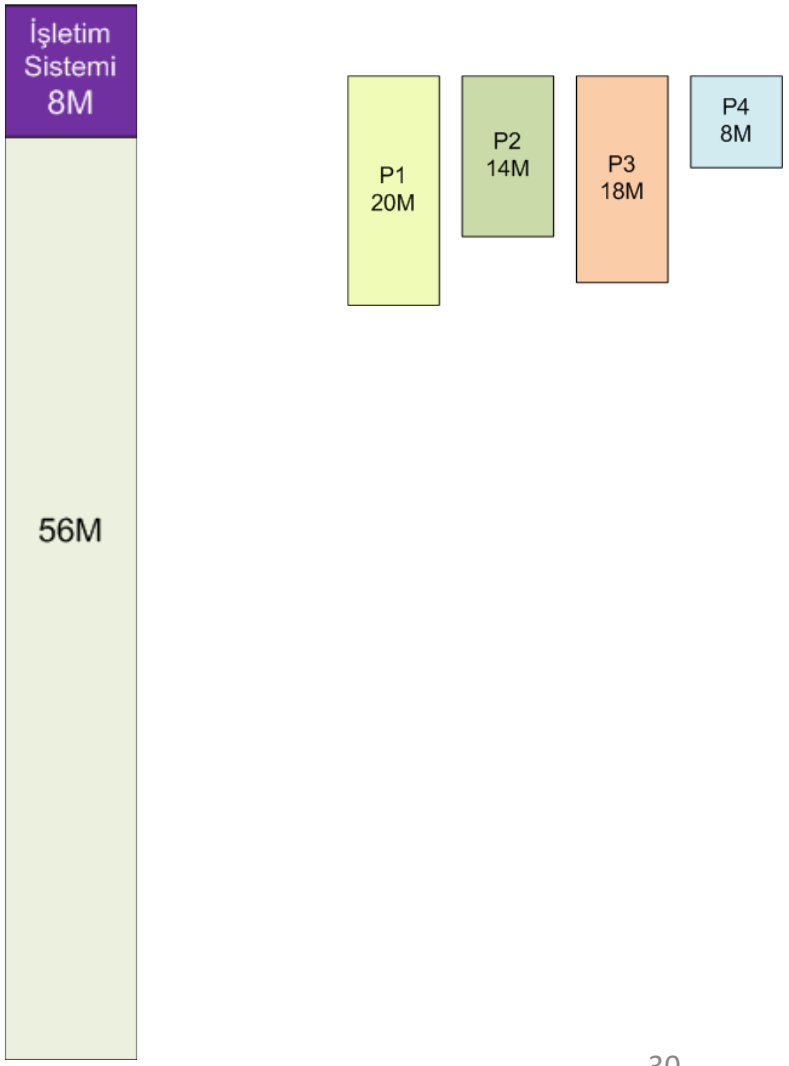
Dinamik Bölümlü Bellek Yönetimi (Dynamic Partitioning)

- **Örnek:**
- 64 Megabayt ana belleğin aşağıdaki dört process için kullanılacağını varsayınız.



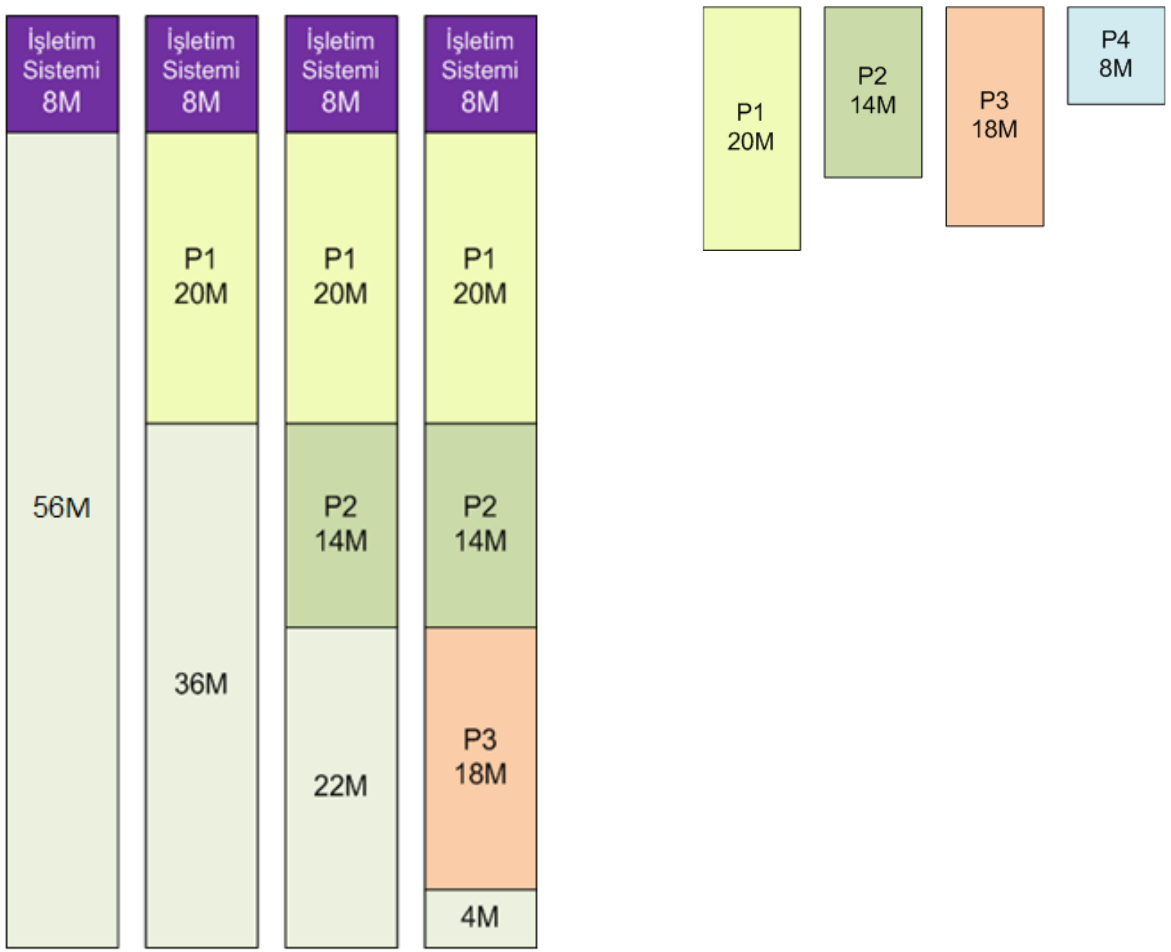
Dinamik Bölümlü Bellek Yönetimi (Dynamic Partitioning)

- Başlangıçta ana belleğin işletim sistemi bölümünün haricindeki alanın boş olduğunu varsayınız.
- Processler birbirleri ardına gerekli bölümleri oluşturarak ana belleğe yüklenirler.



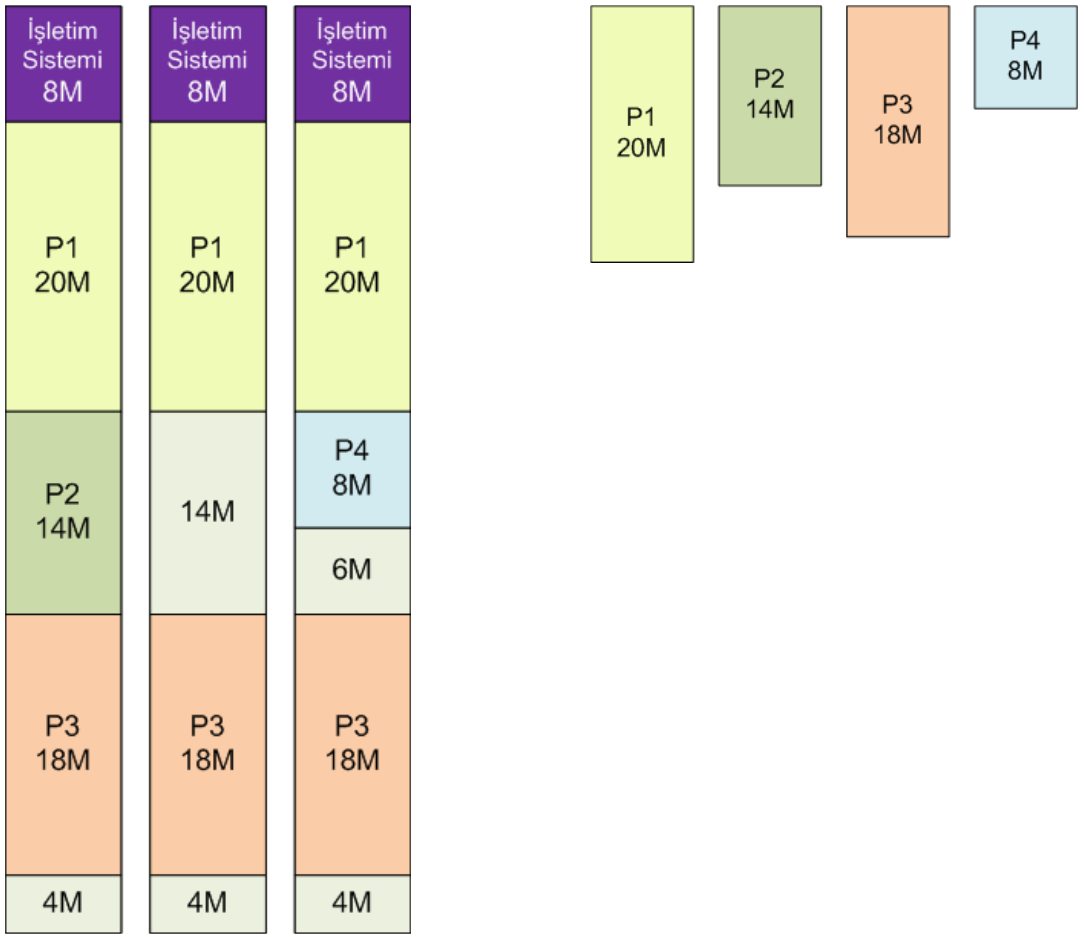
Dinamik Bölümlü Bellek Yönetimi (Dynamic Partitioning)

- P1, P2 ve P3 yerleştirildikten sonra P4 için bellekte yeterli yer yoktur.
- Yer değiştirme (swap) işlemiyle bir process sanal belleğe taşınır.
- P2'nin yer değiştireceğini varsayalım.



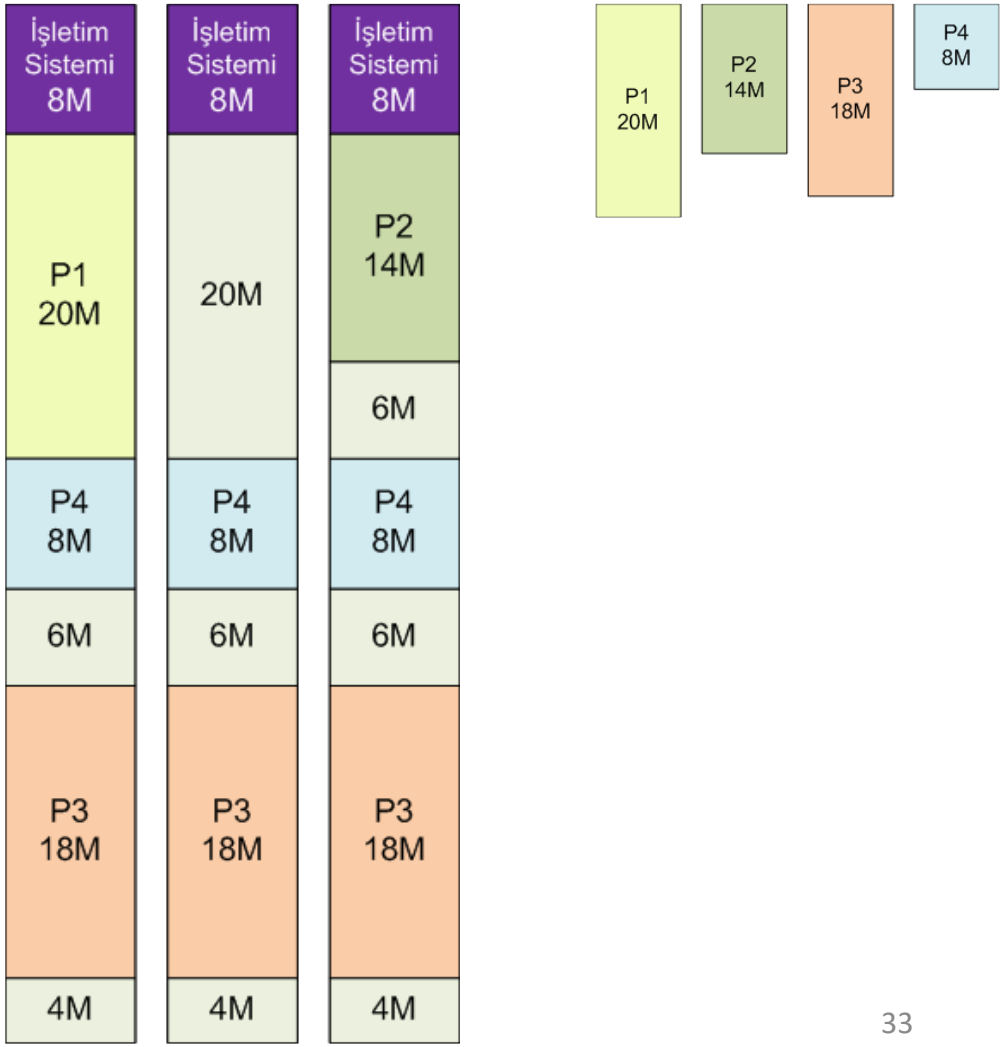
Dinamik Bölümlü Bellek Yönetimi (Dynamic Partitioning)

- P2'nin yer değiştirmesinden sonra bellekte 14 MB'lık bir bölüm serbest kalır.
- Bu bölüme P4 yüklenebilir.



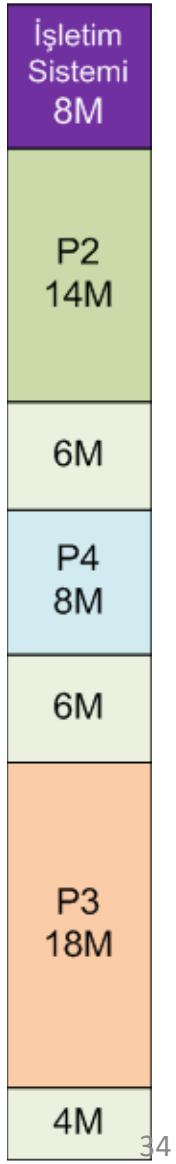
Dinamik Bölümlü Bellek Yönetimi (Dynamic Partitioning)

- P2'nin çalışması için tekrar ana belleğe yüklenmesi gerektiğini varsayalım.
- Bu sefer P1'in bekleme durumunda olduğunu ve yer değiştireceğini varsayalım.
- Yer değiştirme olayından sonra P2 tekrardan ana belleğe yüklenebilir.

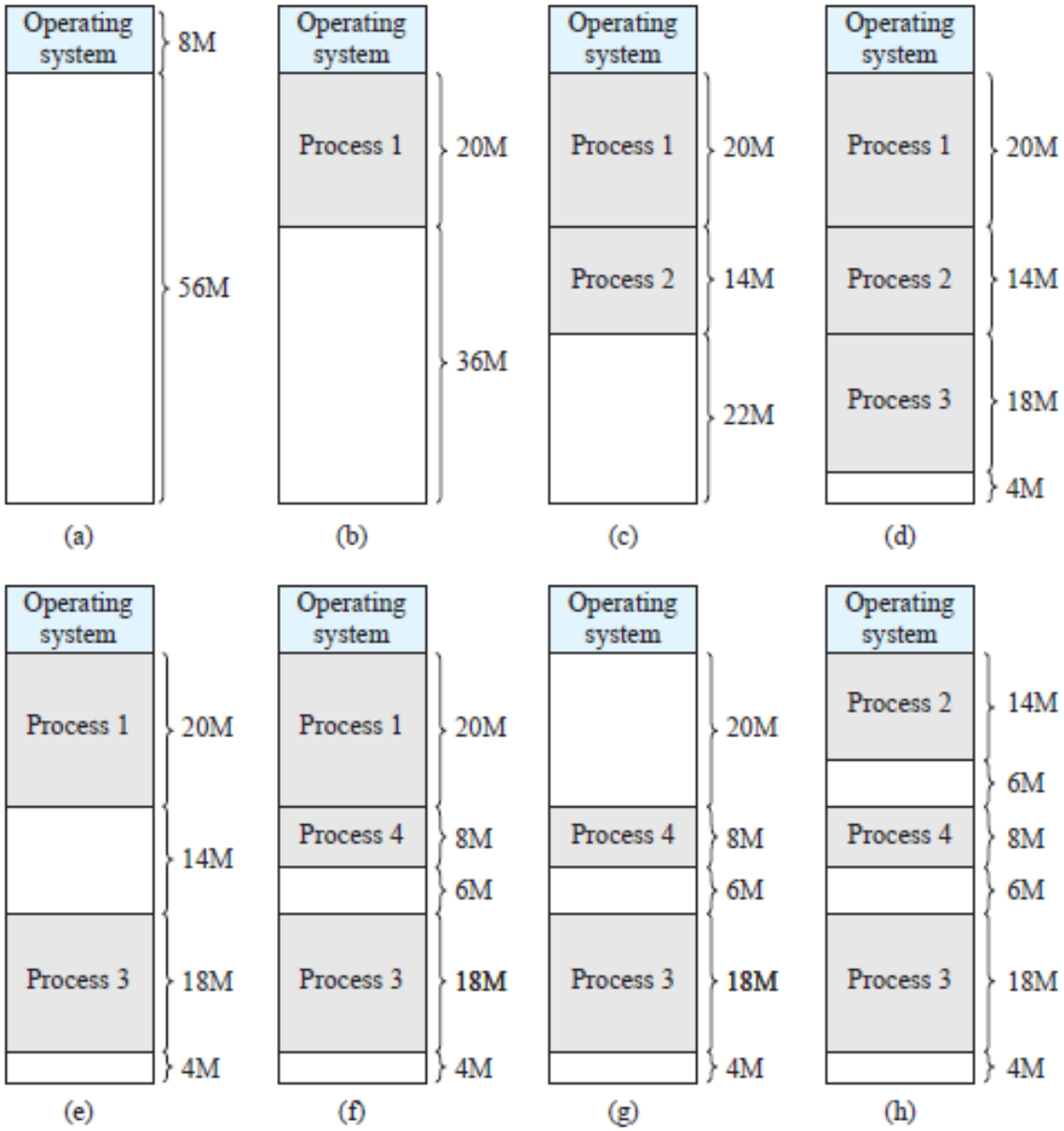


Dinamik Bölümlü Bellek Yönetimi (Dynamic Partitioning)

- **Ana Belleğin Parçalanması Sorunu (dynamic storage allocation problem) :**
- Ana belleğin parçalanması, bitişken alanların process'lere atanan bölümlerle, zaman içinde ufalanması olarak tanımlanır.
- Bu sorun, kullanılan bölümler arasına sıkışmış, işletim için bekleyen process'lerin gereksinimini karşılayamayan boş alanların varlığıyla ortaya çıkar.
- Belleğin parçalanması sonucu, bellekteki boş alanların toplamı, gerekli sığmaları karşılıyor olmasına karşın yeni process'lere yer sağlanamaz durumlarla karşılaşılır.

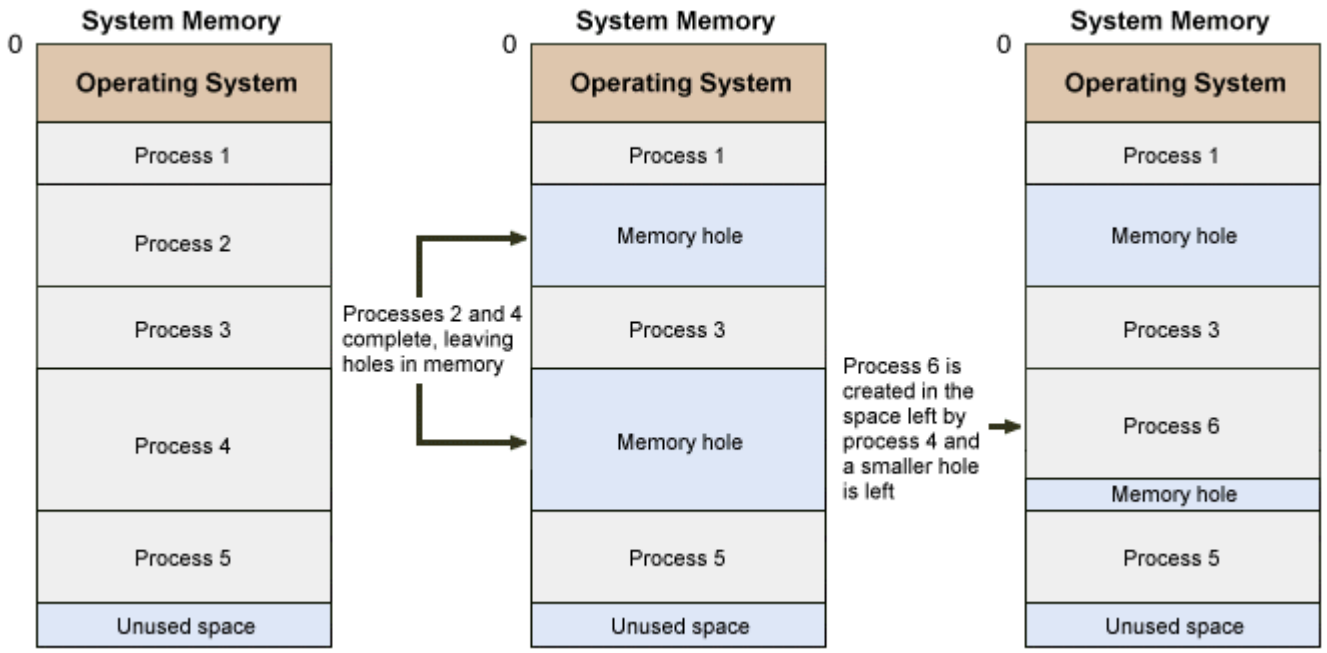


Dinamik Bölümlü Bellek Yönetimi (Dynamic Partitioning)



Dinamik Bölümlü Bellek Yönetimi (Dynamic Partitioning)

- Process'lere sağlanan alanların konumlarının işletim sırasında değiştirilememesi parçalanma sorununun temel nedenidir.
- Dinamik bölümlü bellek yönetiminde bölümler arasında boş alanlar kalmakta, yani dış parçalanma (external fragmentation) oluşmaktadır.

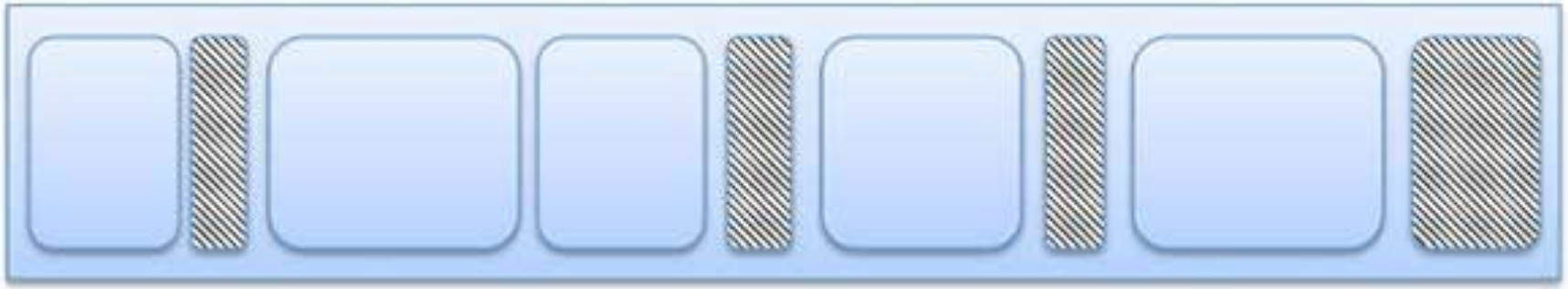


Bitiştirme (Compaction)

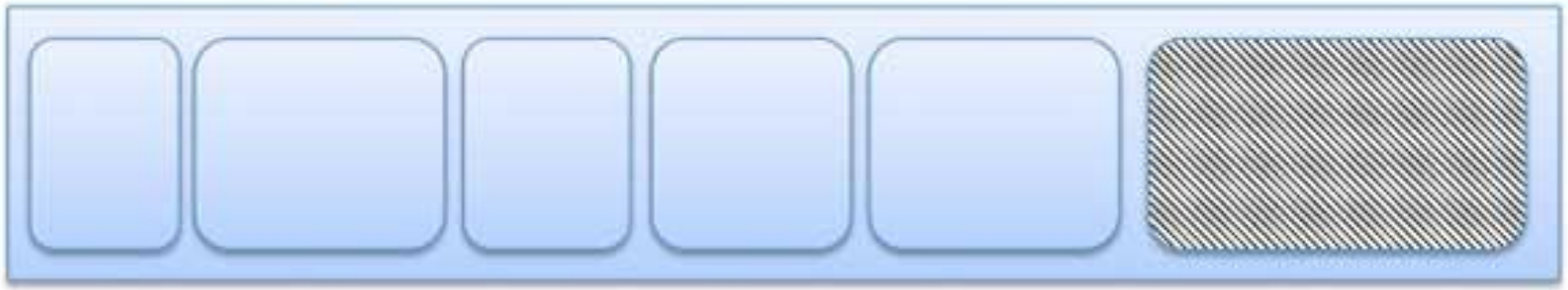
- Dinamik bölümlü bellek yönetiminde bölümler arasında boş alanlar kalmakta, yani dış parçalanma (external fragmentation) oluşmaktadır.
- Process'lere sağlanan bellek konumlarının işletim sırasında değiştirilememesi bitişik alanların parçalanma sorununun temel nedenidir.
- Bu sakıncalı durumun yok edilebilmesi için bitiştirme işlemi (compaction) kullanılabilir.
- Tüm bellek alanına dağılmış durumdaki bölümleri, yerlerini değiştirerek yan yana yerleştirip tek bir bitişken boş alan oluşturma işlemine **bitiştirme işlemi (compaction)** denir.

Bitiştirme (Compaction)

Fragmented memory before compaction

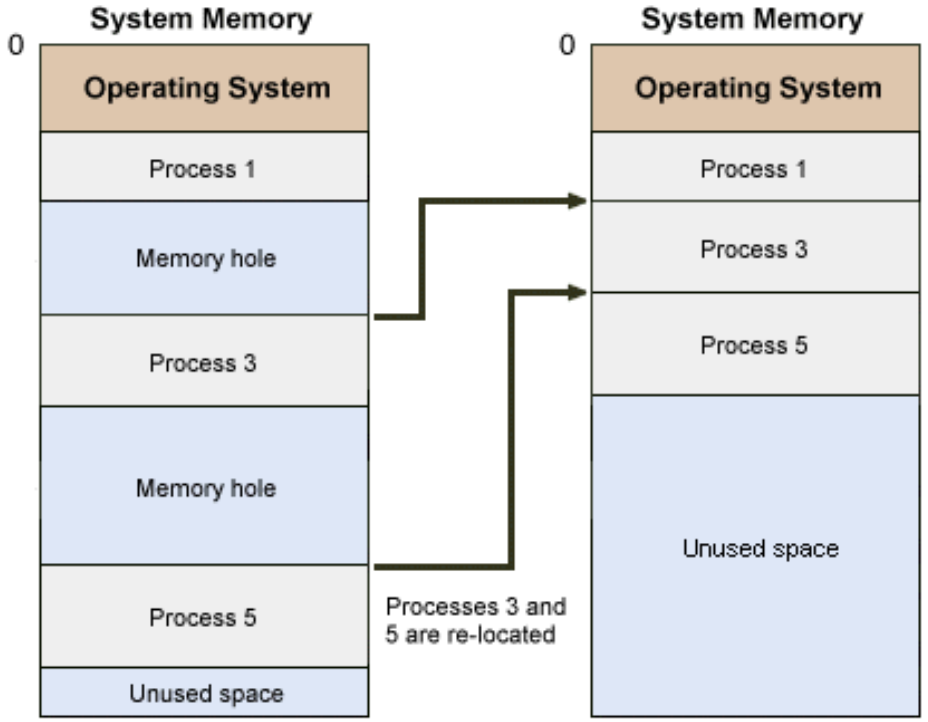


Memory after compaction



Bitiştirme (Compaction)

- Bitiştirme işlemiyle tüm kullanılmayan bölümler bir yerde toplanır.
- Bitiştirme işlemi, işlemciyi meşgul eden, zaman alan ve dolayısıyla bilgisayarı yavaşlatan bir işlemdir.
- Bitiştirme ihtiyacını azaltmak için farklı **yerleştirme algoritmaları** kullanılır.



Dinamik Bölümlü Bellek Yönetimi Yerleştirme Algoritmaları

- İşletim sistemi, bellekteki boşlukların bulunduğu listeden process için en uygun boşluğu belirlemede çeşitli algoritmalar kullanır:

- **İlk Uygun Yer Algoritması (First Fit Algorithm)**
- **Sonraki Uygun Yer Algoritması (Next Fit Algorithm)**
- **En Uygun Yer Algoritması (Best Fit Algorithm)**
- **“Buddy” Yöntemi**

Dinamik Bölümlü Bellek Yönetimi Yerleştirme Algoritmaları

- First-fit algoritması en basit ve en hızlı çalışan algoritmadır.
- Next-fit, first-fit'e göre biraz daha kötüdür, daha fazla bitiştirme (compaction) gerektirir.
- Best-fit, isminin aksine en kötü performansa sahiptir. Process yerleştirildiğinde geriye kalan boş alan pek kullanılamayacağından çok daha sık bitiştirme (compaction) yapılmalıdır.

Dinamik Bölümlü Bellek Yönetimi Yerleştirme Algoritmaları

- **İlk Uygun Yer Algoritması (First-Fit) :**
- Bir process bellek isteğinde bulunduğunda, bellek baştan sona taranır ve process için gerekli olan bellek boyutunu sağlayan ilk boş alana yerleştirilir.

- Gerçekleştirmesi kolay ve en hızlı çalışan algoritmadır.

Dinamik Bölümlü Bellek Yönetimi Yerleştirme Algoritmaları

- **Sonraki Uygun Yer Algoritması (Next-Fit) :**
- İlk uygun yer (First Fit) algoritması ile aynı mantıkta çalışır, fakat bu algoritma en son bulduğu uygun yer bilgisini saklar.
- Bir sonraki aramada belleğin en başından değil de, saklamış olduğu noktadan (bir önce bulduğu uygun yer) itibaren aramaya başlar.
- Genellikle atamalar belleğin son kısımlarında yer alan büyük boşluklardan seçilir.
- Bu nedenle büyük boşluklar küçük parçalara bölünür. Bu da bitiştirme (compaction) gerektirir.

Dinamik Bölümlü Bellek Yönetimi Yerleştirme Algoritmaları

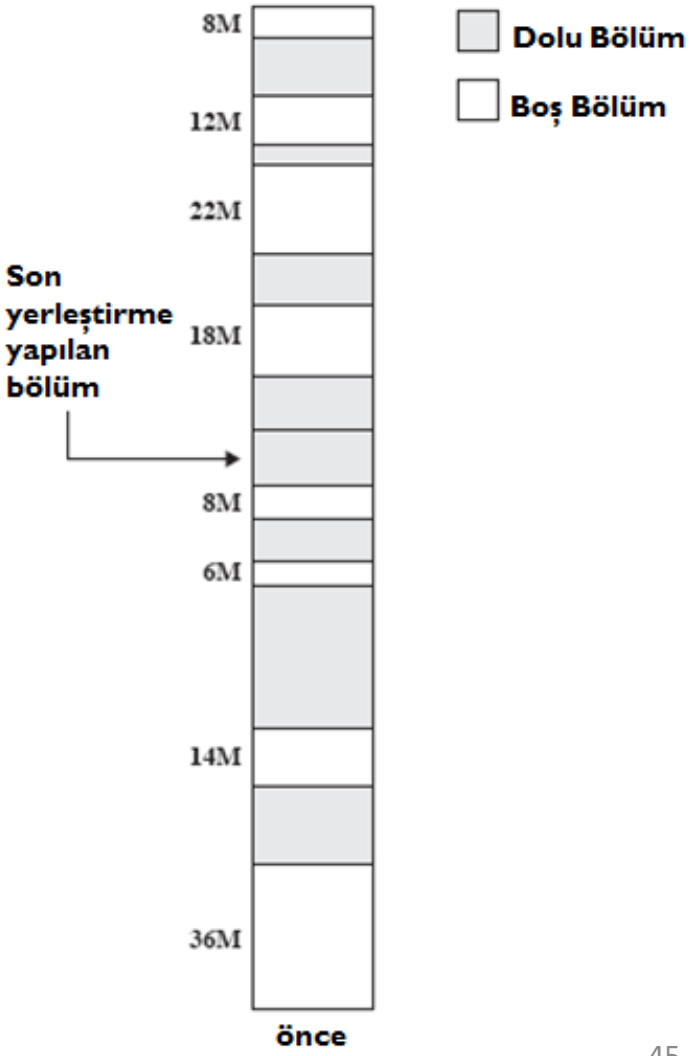
- **En Uygun Yer Algoritması (Best Fit) :**
- Bu algoritma belleğin başından sonuna kadar tüm boş alanları tarar ve process için gerekli olan bellek boyutuna en uygun olan (en yakın boyuttaki) boş bellek alanına process'i yerleştirir.

- En yavaş çalışan algoritmadır.

- Çok küçük ve kullanışsız parçalanmalara sebep olur.

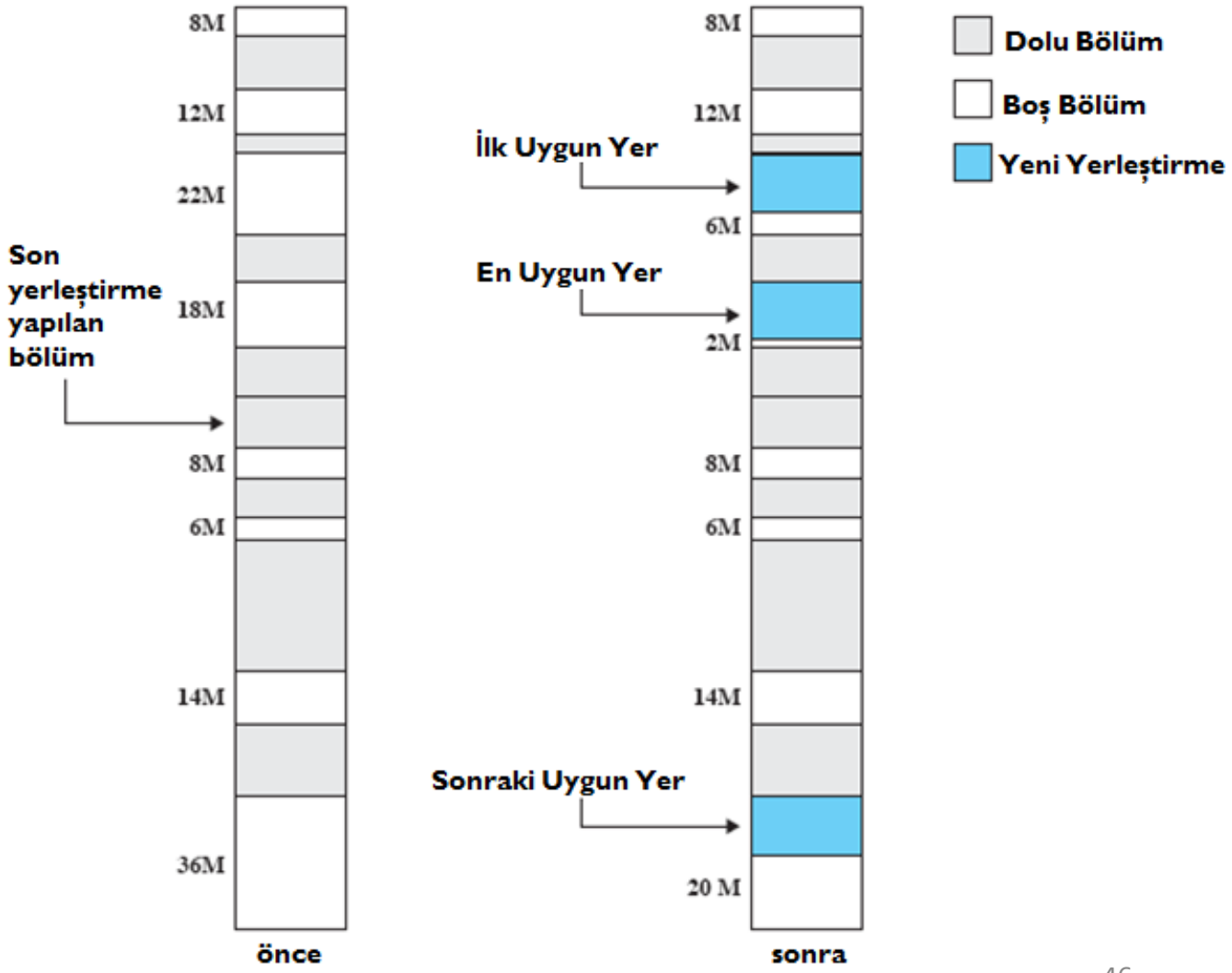
Dinamik Bölümlü Bellek Yönetimi Yerleştirme Algoritmaları

- **Örnek 1**
- Ana bellekteki boş ve dolu bölümlerin yan tarafta göründüğü gibi olduğunu varsayınız.
- 16 MB'lık bir yerleştirme isteğini ilk uygun (first fit), sonraki uygun (next fit) ve en uygun (best fit) yer algoritmasına göre belleğe yerleştiriniz.



Dinamik Bölümlü Bellek Yönetimi Yerleştirme Algoritmaları

- İlk uygun yer (first fit) algoritması 22 MB'lık bölümü kullanarak 6 MB'lık dış parçalanma,
- Sonraki uygun yer (next fit) algoritması son 36 MB'lik bölümü kullanarak 20 MB'lik dış parçalanma,
- En uygun yer (best fit) algoritması da tüm bölümleri tarayarak 18 MB'lık bölümü uygun görür. Buranın kullanımı ile, 2 MB'lık dış parçalanma oluşturur.



Dinamik Bölümlü Bellek Yönetimi Yerleştirme Algoritmaları

- **Örnek 2**
- Ana bellekte sırayla aşağıdaki boş bölümlerin olduğunu varsayınız.
- 24 KB, 20 KB ve 18 KB'lık bellek kullanım istekleri için sırasıyla hangi boşlukların kullanılacağını ve oluşacak parçalanmaları aşağıdaki algoritmaları kullanarak belirtiniz.
- İlk Uygun Yer (First Fit) Algoritması
- Sonraki Uygun Yer (Next Fit) Algoritması
- En Uygun Yer (Best Fit) Algoritması

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3	BOŞLUK 4	BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	40 K	36 K	14 K	18 K	24 K	30 K

Dinamik Bölümlü Bellek Yönetimi Yerleştirme Algoritmaları

- İlk Uygun Yer (First Fit) Algoritması

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3	BOŞLUK 4	BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	40 K	36 K	14 K	18 K	24 K	30 K

24 K Boşluk 3 (16K parçalanma)

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3		BOŞLUK 4	BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	24 K	16 K	36 K	14 K	18 K	24 K	30 K

20 K Boşluk 1 (0K parçalanma)

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3		BOŞLUK 4	BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	24 K	16 K	36 K	14 K	18 K	24 K	30 K

18 K Boşluk 4 (18K parçalanma)

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3		BOŞLUK 4		BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	24 K	16 K	18 K	18 K	14 K	18 K	24 K	30 K

Dinamik Bölümlü Bellek Yönetimi Yerleştirme Algoritmaları

- Sonraki Uygun Yer (Next Fit) Algoritması

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3	BOŞLUK 4	BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	40 K	36 K	14 K	18 K	24 K	30 K

24 K Boşluk 3 (16K parçalanma)

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3		BOŞLUK 4	BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	24 K	16 K	36 K	14 K	18 K	24 K	30 K

20 K Boşluk 4 (16K parçalanma)

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3		BOŞLUK 4		BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	24 K	16 K	20 K	16 K	14 K	18 K	24 K	30 K

18 K Boşluk 6 (0K parçalanma)

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3		BOŞLUK 4		BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	24 K	16 K	20 K	16 K	14 K	18 K	24 K	30 K

Dinamik Bölümlü Bellek Yönetimi Yerleştirme Algoritmaları

- En Uygun Yer (Best Fit) Algoritması

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3	BOŞLUK 4	BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	40 K	36 K	14 K	18 K	24 K	30 K

24 K Boşluk 7 (0K parçalanma)

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3	BOŞLUK 4	BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	40 K	36 K	14 K	18 K	24 K	30 K

20 K Boşluk 1 (0K parçalanma)

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3	BOŞLUK 4	BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	40 K	36 K	14 K	18 K	24 K	30 K

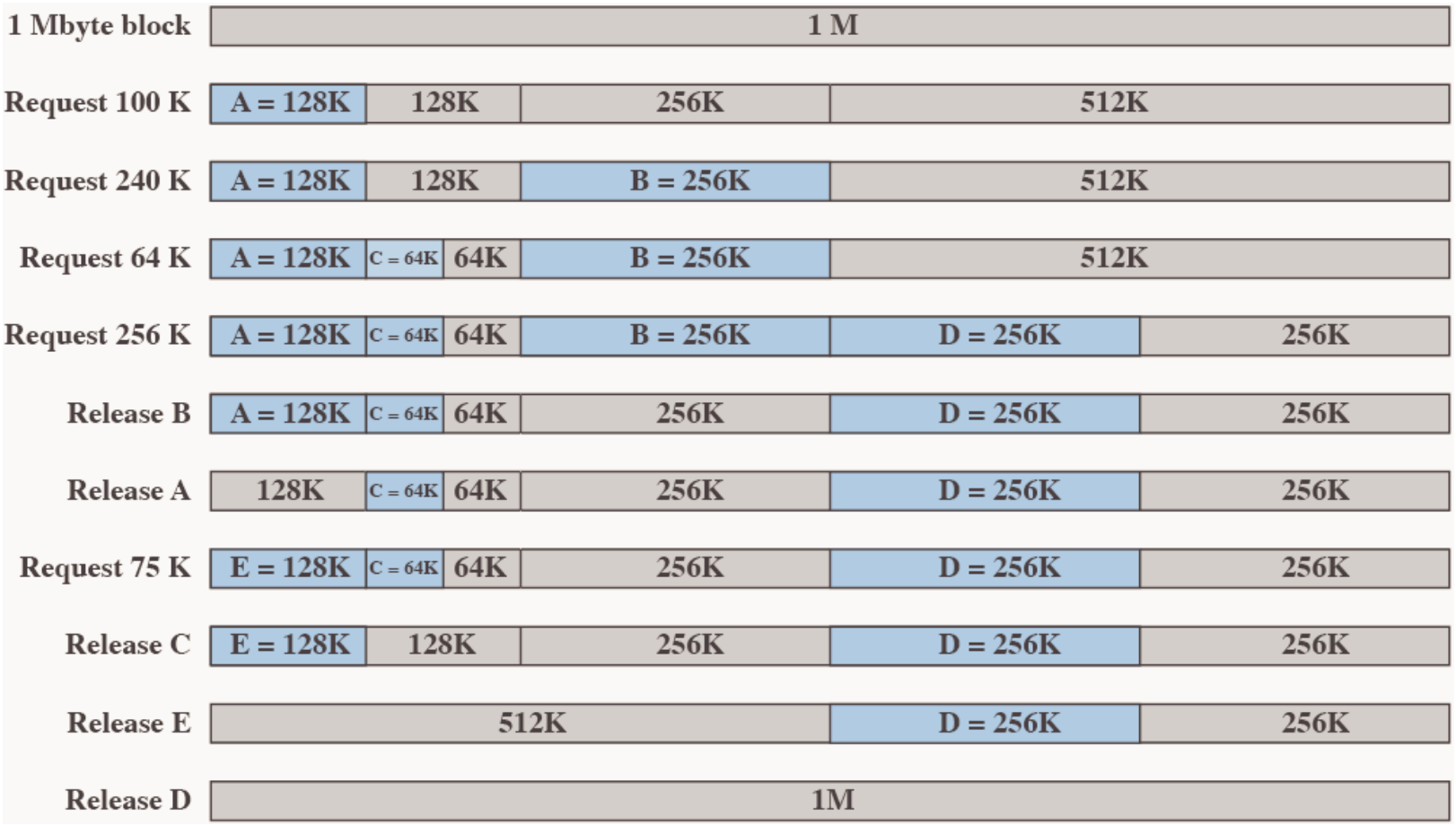
18 K Boşluk 6 (0K parçalanma)

BOŞLUK 1	BOŞLUK 2	BOŞLUK 3	BOŞLUK 4	BOŞLUK 5	BOŞLUK 6	BOŞLUK 7	BOŞLUK 8
20 K	8 K	40 K	36 K	14 K	18 K	24 K	30 K

“Buddy” Yöntemi

- Tüm boş alan 2^U boyutunda tek bir alan olarak ele alınır.
- Buddy sistemi bellek bölümlerini 2^i olarak ayırır. i=0,1,2,3,….
- Bellek bölümleri örneğin, 256, 64, 32 … şeklinde olabilir.
- S boyutundaki bir istek eğer $2^U-1 < S <= 2^U$ ise tüm blok atanır.
- Aksi halde blok 2^U-1 boyutunda iki eş bloğa bölünür (buddy)
- S’den büyük veya eşit en küçük birim blok oluşturulana kadar işlem devam eder.

“Buddy” Yöntemi



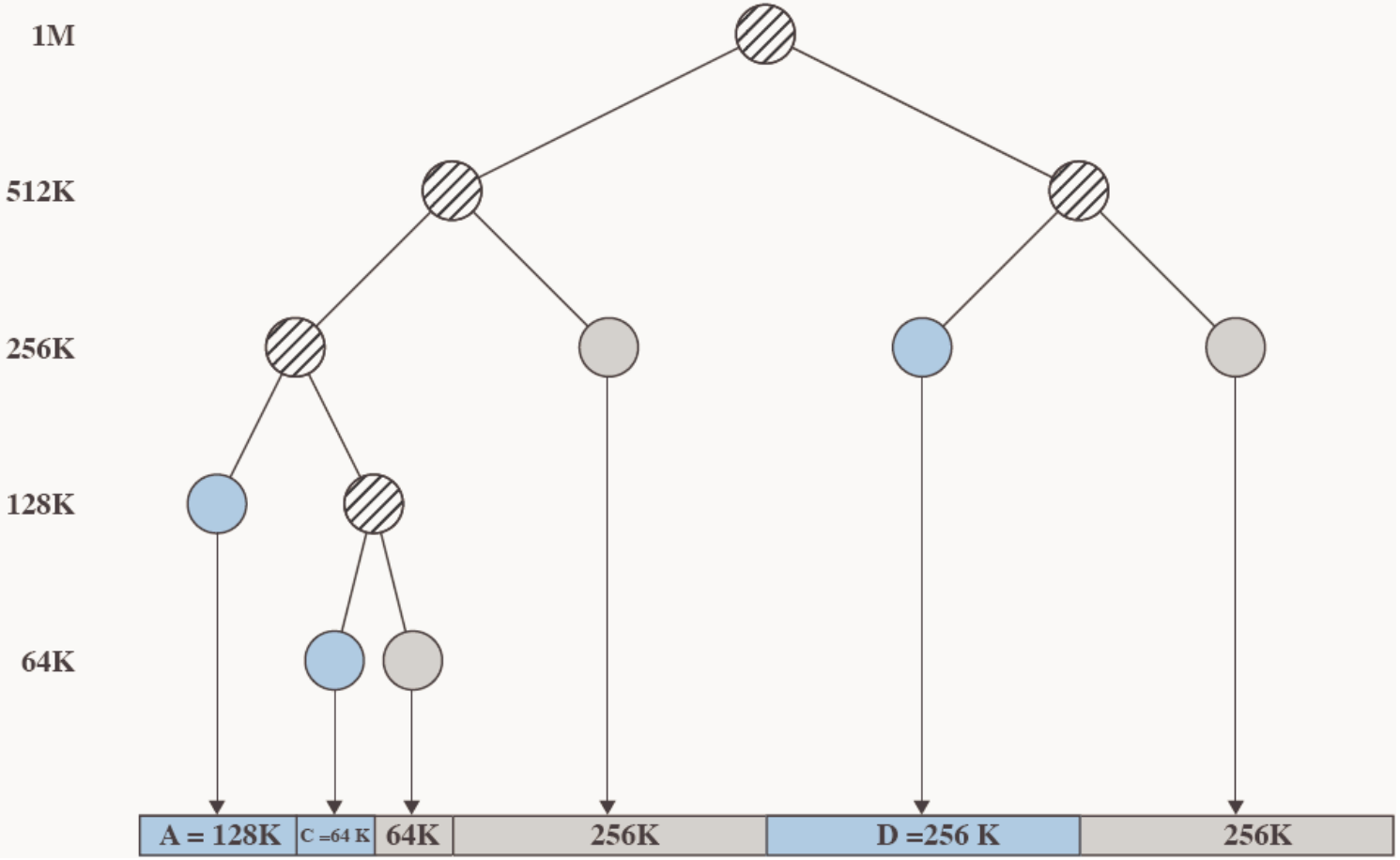
Başlangıç 1 Mbyte ilk blok.

İlk istek A, 100 Kbyte'dır ve 128 KB bloğa ihtiyaç var . İlk blok iki 512 KB bölüme ayrılır. Bunların birincisi iki 256 KB bölüme ayrılır ve bunların ilki, biri A'ya atanmak üzere iki 128 KB bölüme ayrılır.

Bir sonraki istek B, 240 Kbyte'dır ve 256 KB blok gerektirir. Böyle bir blok zaten mevcut ve tahsis edilmiştir.

Süreç, gerektiğinde meydana gelen bölme ve kaynaşma ile devam eder.

“Buddy” Yöntemi



Parçalanma (Fragmentation)

- Programlar belleğe yüklenirken ve atılırken bellek alanları sürekli parçalanır (fragmentation).
- Bir process için yeterli alan olabilir, ancak bunlar küçük parçalar halinde dağılmış durumda olabilir.
- En kötü durumda her iki process arasında boş kısım olabilir.
- First fit ile yapılan istatistiksel analize göre, **N** tane kullanılmış bölüm için **N/2** tane boş bölüm oluşur.
- Fragmentation çözümünde küçük bölümler yer değiştirilerek bitiştirme yapılabilir (compaction) fakat fazla süre gerektirir ve performansı düşürür.
- **Sayfalı bellek (paging) ve Kesimli bellek (Segmentation) yönetimleri parçalanma (fragmentation) çözümünde etkindir.**

Mantıksal vs. Fiziksel Adresler

- Bir program, farklı bellek kapasiteli sistemler üzerinde çalıştırılabilir.
- Yazılımcı geliştirdiği bir programın çalıştırıldığında fiziksel bellekte nereye yerleşeceği ile ilgilenmeden uygulamasını geliştirir.
- Bu yüzden yazılımcı sanki tüm bellek uzayı sadece kendisine aitmiş gibi kod yazar.
- Yazılımcının bir programlama dili ile yazmış olduğu programlara **kaynak program (source program)** denir.
- Kaynak programın derlenmesi ile elde edilen makine diline dönüştürülmüş programa ise **amaç program (object program)** denir.

Mantıksal vs. Fiziksel Adresler

- Amaç program, tüm adreslerin program başına (sıfıra) göreceli olarak belirlendiği programdır.
- Amaç program içindeki göreceli adresler, mantıksal adresler olarak adlandırılırlar.
- Amaç programlar sadece mantıksal adresleri kullanır, fiziksel bellek adreslerini içermez.
- İşletim sistemi Bellek yöneticisinin görevlerinden biri de mantıksal adresleri fiziksel bellek adres değerlerine dönüştürmektir.

Mantıksal vs. Fiziksel Adresler

- **Fiziksel adres (Physical address) :**
- Fiziksel bellek adresleri, işlemcinin doğrudan eriştiği, ana bellek içindeki gerçek fiziksel konumları temsil eder.
- Fiziksel adresler, bellek hücrelerini veya bellek bloklarını işaret eder.
- Fiziksel bir adres, belleğin yerini belirlemek için kullanılırken, mantıksal adres ise fiziksel bir adrese erişmek için kullanılır.

Mantıksal vs. Fiziksel Adresler

- **Mantıksal adres (Logical/Virtual address) :**
- Mantıksal bellek adresleri, bir process tarafından kullanılan adreslerin mantıksal tanımlamalarıdır.

- Bu adres, process'in yürütülmesi sırasında işlemci tarafından fiziksel bellek konumuna erişmek için referans olarak kullanılır.

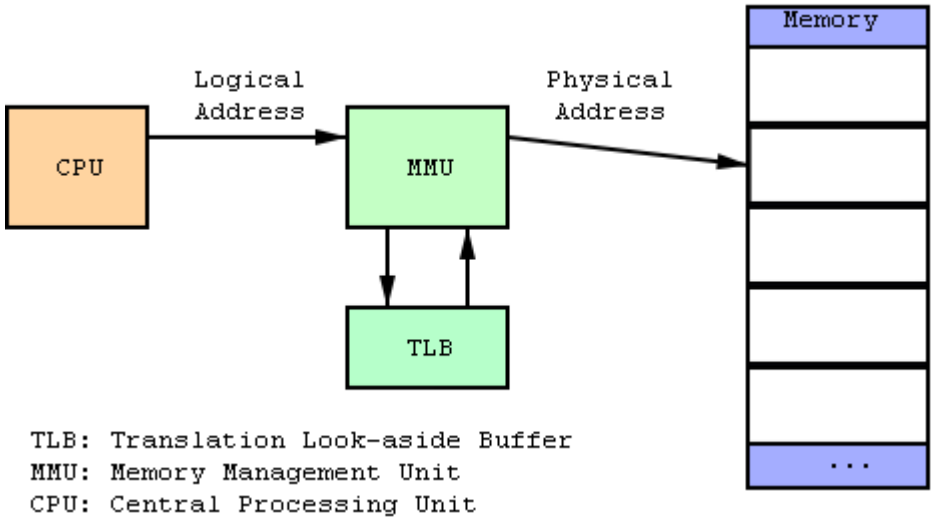
Mantıksal vs. Fiziksel Adresler

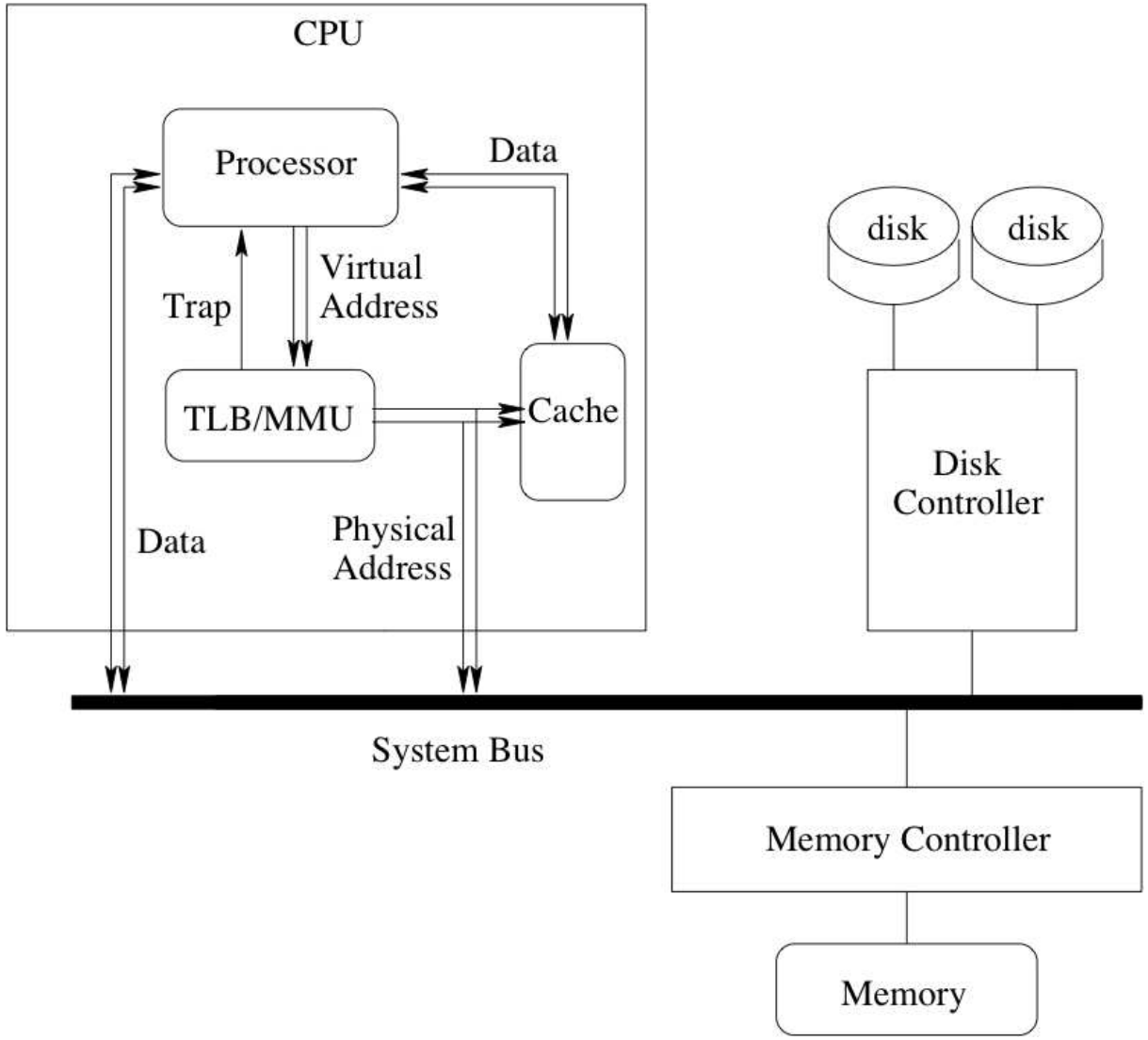
- İşletim sistemi, ana belleğin fiziksel adres alanını yönetir ve process'lerin mantıksal adreslerini fiziksel bellek adreslerine dönüştürür.

- İşlemci içinde bulunan Bellek Yönetim Birimi (Memory Management Unit-MMU) mantıksal adresleri dinamik olarak fiziksel adreslere eşleştirmede yararlanılan donanım birimidir.

- Mantıksal adres, process yürütülürken CPU tarafından üretilirken, fiziksel adres Bellek Yönetim Birimi (MMU) tarafından hesaplanır.

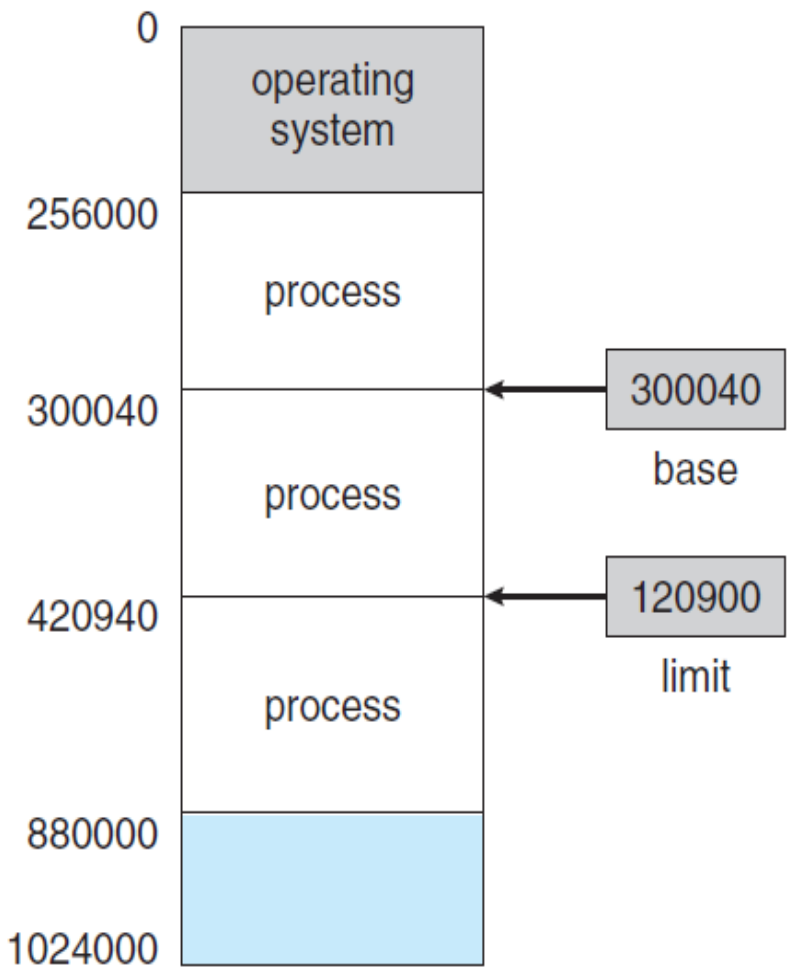
Mantıksal vs. Fiziksel Adresler





Bitişik bellek ayırma (Contiguous memory allocation) için Mantıksal vs. Fiziksel Adresler

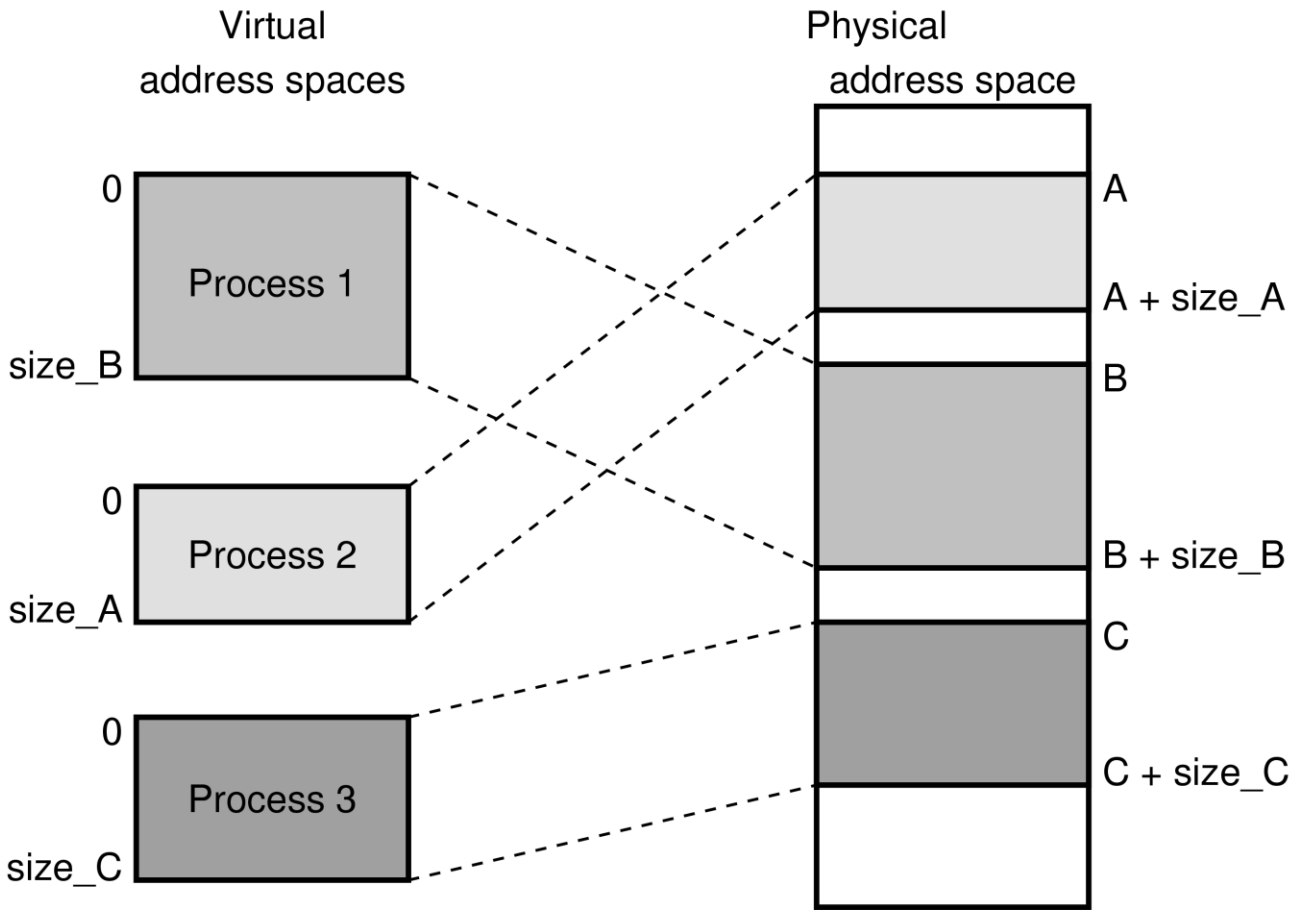
- Bitişik bellek ayırma tekniğinde fiziksel ana bellek process'lerinin yerleşimi için bölmelere ayrılır.
- Bellekteki bu bölmelerin başlangıç adresleri (base address) ve bitiş adresleri (limit address) belirlidir.
- Belleğe process'lerin yerleşimi Base register ve Limit register tarafından sağlanan adresler sayesinde gerçekleştirilir.



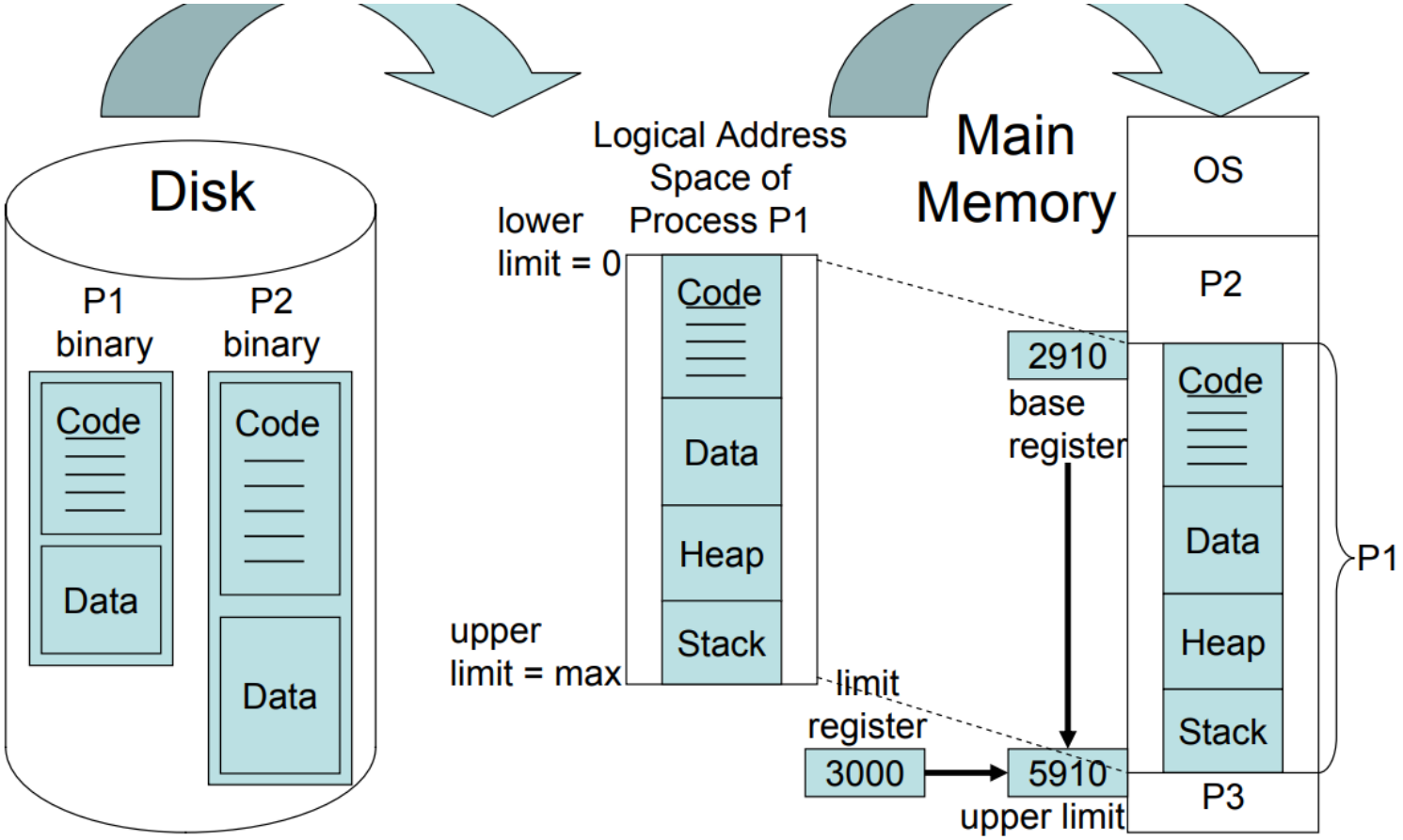
Bitişik bellek ayırma (Contiguous memory allocation) için Mantıksal vs. Fiziksel Adresler

- **Base register aynı zamanda relocation register olarak da bilinir.**
- Base (relocation) register process için ayrılan fiziksel adres bölümünün taban değerini içerir.
- Limit register, process'in mantıksal adres aralığının uzunluk değerini içerir. Başka bir deyişle, mantıksal adres alanını tanımlar.
- Her mantıksal adres limit register değerinden daha küçük olmalıdır.
- Fiziksel adres: Base (relocation) register + Mantıksal adres
- Mantıksal adres uzayı, aralık (0 – max)
- Fiziksel adres uzayı, aralık (R+0 – R+max)
- R : Relocation register
- CPU, 0 ila max aralığında mantıksal adres üretebiliyorsa, ilgili fiziksel adres R + 0'dan R + max kadar değişebilir.
- Base (relocation) register ve limit register değerleri toplanarak process için ayrılan bellek bölümün bittiği adres bulunabilir.

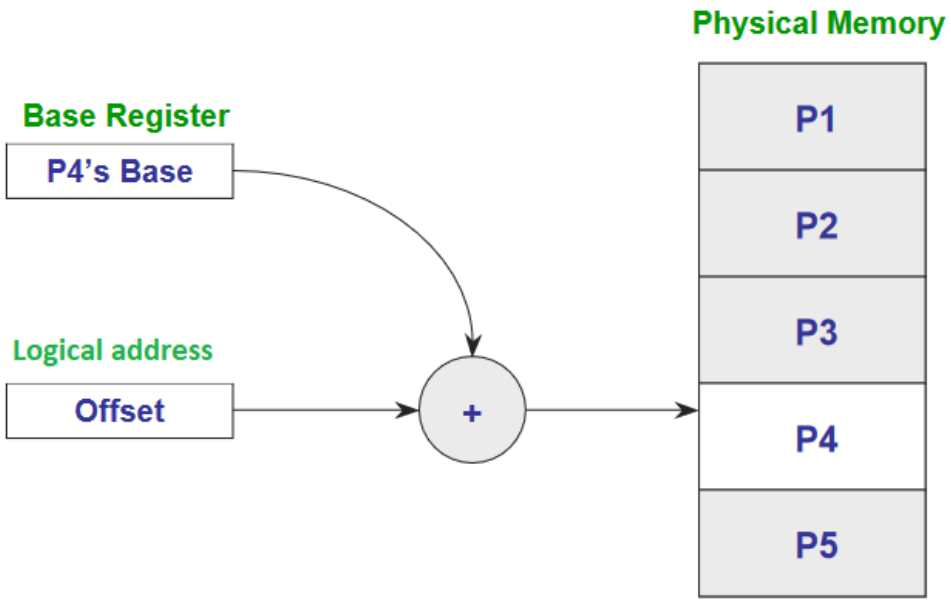
Bitişik bellek ayırma (Contiguous memory allocation) için Mantıksal vs. Fiziksel Adresler



Bitişik bellek ayırma (Contiguous memory allocation) için Mantıksal vs. Fiziksel Adresler



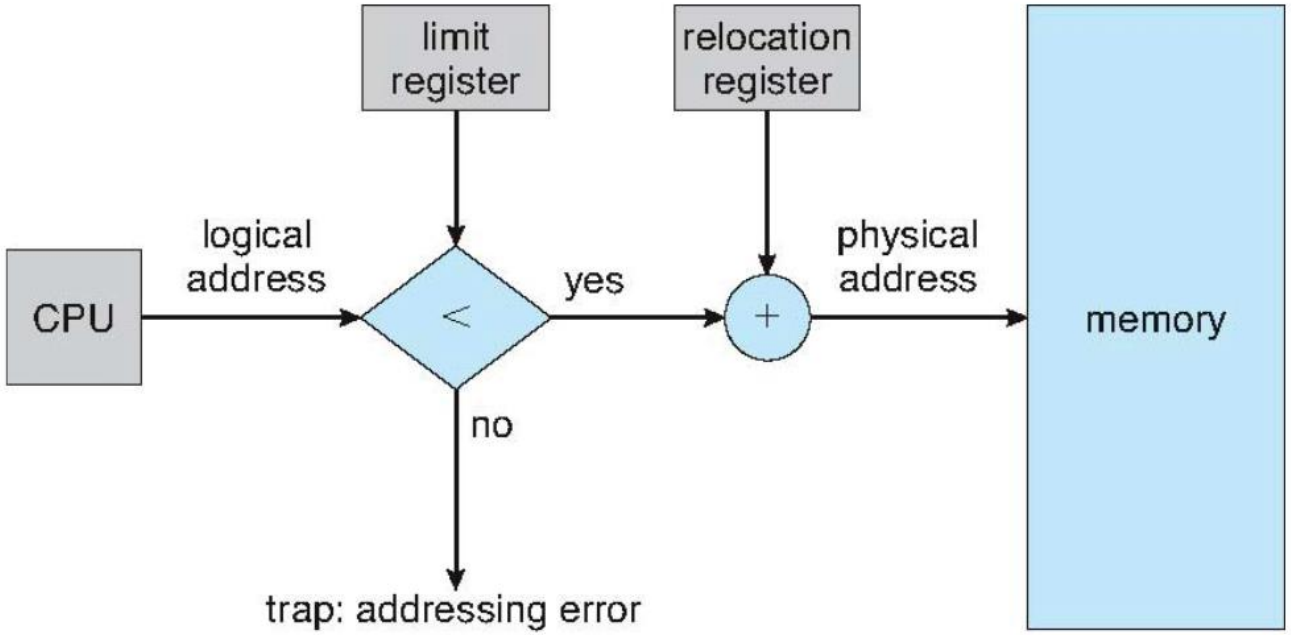
eşit uzunlukta bellek bölümleri (equal-size partitioning)
Mantıksal Adresten Fiziksel Adres Dönüşümü

- Fiziksel bellek, sabit bölümlere ayrılmıştır. Her bölümün boyutu aynı ve sabittir.
- Mantıksal adresler process başına göreceli (offset) olarak belirlenir. Eşit uzunlukta bellek bölümleri için mantıksal adres sadece offset değerinden oluşur.
- 

- **Physical address = base (relocation) register + logical address**

Eşit-olmayan uzunlukta bellek bölümleri (unequal-size partitioning)

Mantıksal Adresten Fiziksel Adres Dönüşümü



- CPU tarafından oluşturulan mantıksal adres (Logical address) limit register ile karşılaştırılır ve değer düşükse, Fiziksel adres (Physical address) oluşturmak için mantıksal adres relocation registerdeki değere eklenir.
- Dolaysıyla her mantıksal adres değeri limit register değerinden daha küçük olmalıdır.

Eşit-olmayan uzunlukta bellek bölümleri (unequal-size partitioning)

Mantıksal Adresten Fiziksel Adres Dönüşümü

- Memory Management Unit (MMU) planında,
- Bir kullanıcı uygulaması işlemcide yürütülüyorken her mantıksal adrese (logical/virtual address), relocation register'deki değer eklenir ve fiziksel adres elde edilir.

1) CPU, Logical address oluşturur

2) Logical address, Limit register değeri ile karşılaştırılır.

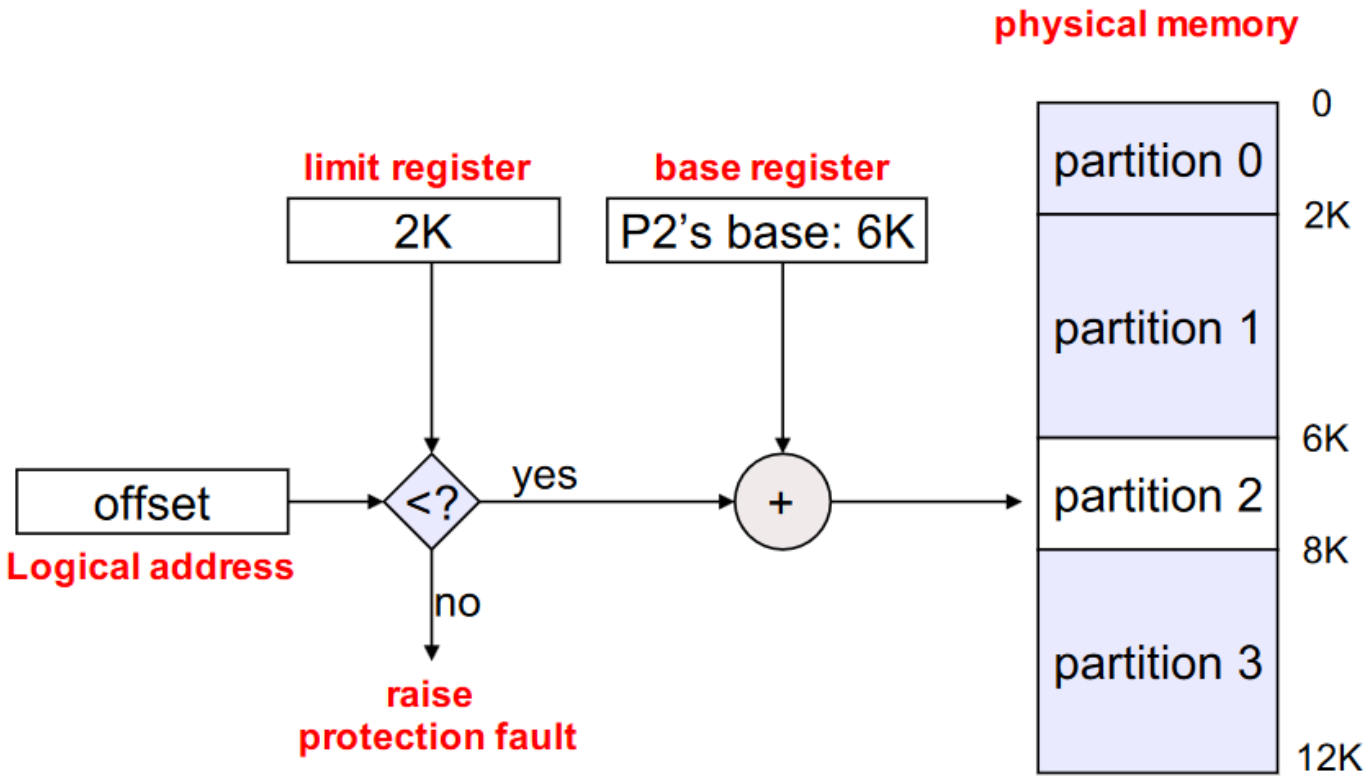
3) Eğer Logical address > Limit register ise ardından TRAP (bellek hatası)

4) Eğer Logical address < Limit register ise,
Physical address = Relocation register + Logical address

Eşit-olmayan uzunlukta bellek bölümleri (unequal-size partitioning)

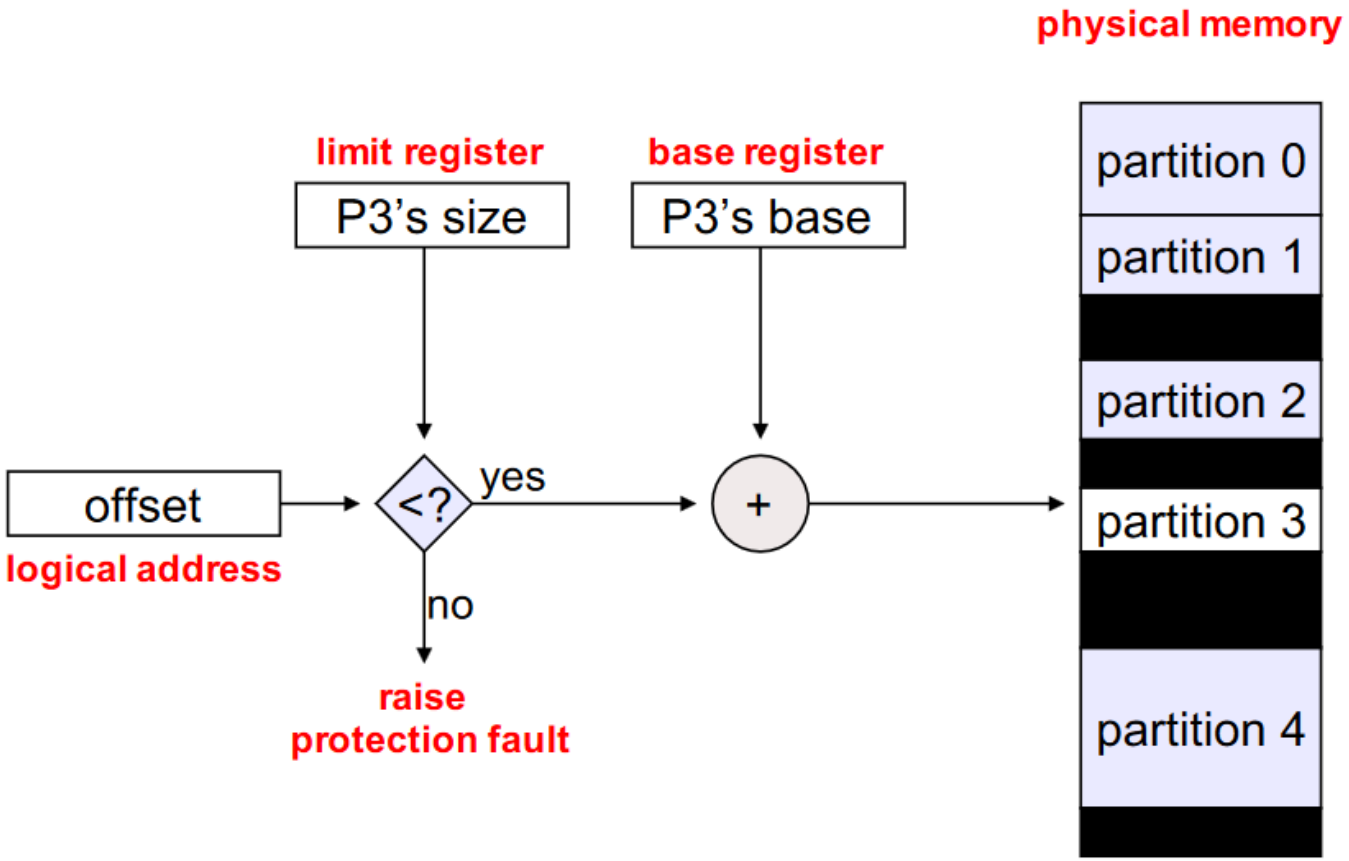
Mantıksal Adresten Fiziksel Adres Dönüşümü

Fiziksel bellek, eşit olmayan bölümlere ayrılmıştır. Her bölümün boyutu farklıdır.



Physical address = base (relocation) register + logical address

Dinamik Bölümlü Bellek (Dynamic partitioning)
Mantıksal Adresten Fiziksel Adres Dönüşümü



physical address = base (relocation) register + logical address